Design Document

Professor	*University Course*
Nome Professore	Nome del corso

Author	*Personal Code*
Nome 1	Matricola 1
Nome 2	Matricola 2
Nome 3	Matricola 3

Settembre, 2025

Contents

1. Introduction

1.1 Purpose

Triplo is a cross-platform mobile application for smartphones and smartwatches, conceived as a digital logbook for trekking activities.
The application aims to enhance the experience of hiking in mountain environments by combining route information, environmental data, and recording of personal activity.
Users can explore predefined trekking routes and access useful information about the paths, including navigation support and contextual data such as weather conditions.
At the same time, the application allows hikers to document their experiences by creating logbook entries enriched with photos and personal diary notes.
The platform also enables users to discover and follow other hikers, and to browse selected entries from their trekking diaries, enhancing the sharing of experiences and inspiration for future hikes.

During a hike, users can optionally complete challenges, such as time-based goals designed to enrich the trekking experience.
The following sections describe the system architecture, the main functionalities of the application, and the technological choices adopted in its development.

1.2 Scope

Triplo supports users during trekking activities by providing guidance, contextual information, and tools to document their experiences in mountain environments.
The application focuses on the Madonna di Campiglio area and offers a set of predefined trekking routes with different levels of difficulty.
During a hike, users can access information related to the selected route, including navigation support and environmental conditions that may influence the trekking experience.
The application also encourages engagement through optional challenges that users may complete along the route. Each trekking activity can be recorded in a personal digital logbook, where users can document their experience by adding photos and diary notes. Users can also browse selected entries from the logbooks of other hikers they follow, allowing them to discover new routes and gain inspiration from shared experiences.
The application aims to support outdoor exploration while enabling users to record, revisit, and share meaningful moments from their trekking activities.

1.3 Definitions, Acronyms, Abbreviations

- GPS Generic term used in this document to indicate satellite-based positioning systems available on the device (such as GPS, Galileo, and others)

- LTE (Long Term Evolution) is a mobile communication standard. LTE is also referred to as "4G", the fourth generation of mobile communication technology. LTE consists of a number of enhancements to the "Universal Mobile Telecommunications System" (UMTS) – the third generation of mobile communications.

1.4 References

1.4.1 Main References

- Design and Implementation of Mobile Applications Lectures and Slides
  - Introduction https://webeep.polimi.it/mod/resource/view.php?id=399280
  - Flutter https://webeep.polimi.it/mod/resource/view.php?id=399287
  - Android https://webeep.polimi.it/mod/resource/view.php?id=411641
  - iOS https://webeep.polimi.it/mod/resource/view.php?id=417791
  - Lectures https://webeep.polimi.it/mod/url/view.php?id=376880

1.4.2 Other References

- Flutter Documentation https://docs.flutter.dev/
- Firebase Documentation https://firebase.google.com/docs/
- OpenWeather API Documentation
  https://openweathermap.org/api/one-call-3?collection=one_call_api_3.0
- Severe Weather Alerts API
  https://www.weatherbit.io/api/alerts
- Oracle Documentation: What Is Model-View-Controller (MVC)?
  https://www.oracle.com/technical-resources/articles/javase/application-design-with-mvc
  html
- Deutsche Telekom Documentation: What is LTE?
  https://www.telekom.com/en/company/details/the-nine-most-important-facts-about-lte-606

2. Architectural Design

2.1 Overview

2.1.1 High-level components

The Triplo system is designed according to a modular architecture in which the main application logic is implemented on the client side, while backend functionalities are delegated to managed services and external APIs, as illustrated in Figures 2.1 and 2.2.

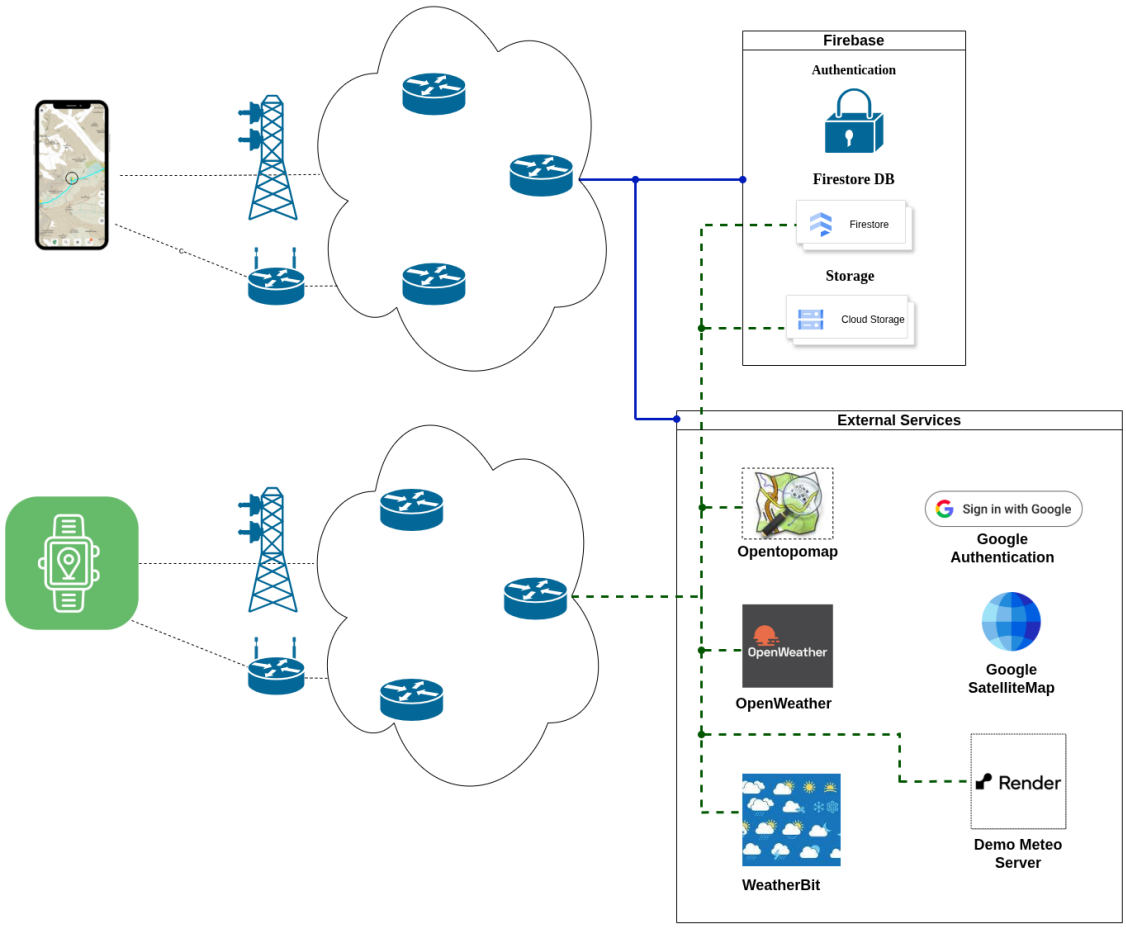


Figure 2.1: Overview Diagram: Blue lines denote the smartphone application's connectivity with Firebase and external services, providing full access to all Firebase modules and external APIs. Dashed green lines denote the smartwatch application's connectivity, which is limited to selected Firebase services, Firestore and Storage and specific external services, for example OpenTopoMap. The smartphone and smartwatch applications access the network independently, reflecting their ability to operate as standalone clients, while still relying on a shared set of backend services for data and functionality.

2.1.2 High level components interaction

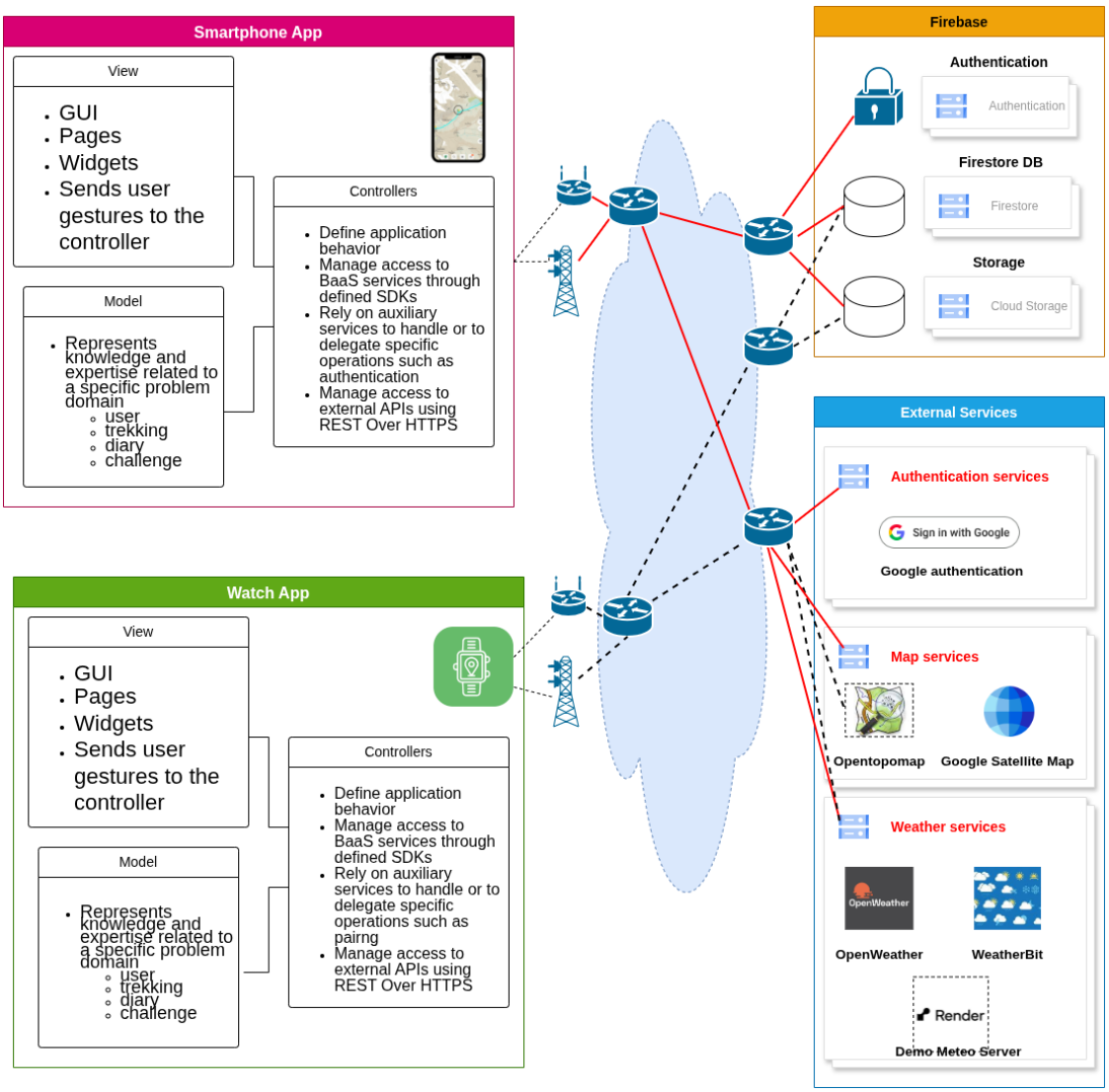


Figure 2.2: Detailed Overview Diagram

The system separates concerns between presentation, application logic, and data management by adopting a Model–View–Controller (MVC) architectural pattern. Both the presentation layer and the core application logic are implemented within the Flutter-based applications, while remaining logically decoupled through the use of Controllers that mediate between Views and Models.
In particular, the mobile application represents the main client, while the smartwatch application provides a lightweight extension offering a subset of functionalities adapted to the constraints and interaction model of wearable devices, relying on the same backend services and data sources.
User interactions are handled by the View layer, while business logic and coordination between components are delegated to Controllers, which act as the central mediation layer between the user interface and the underlying data models.
Data management and authentication are handled through Firebase services, specifically Firebase Authentication, Cloud Firestore, and Firebase Storage. The system leverages Firebase as a Backend-as-a-Service (BaaS) platform, providing managed services for authentication, data storage and user management, relying on platform-specific SDKs for access to these resources. The adoption of Backend-as-a-Service (BaaS) solutions enables the system to replace a traditional custom server-side layer with a more lightweight and scalable architecture. By delegating data management, authentication, and storage to managed services, architectural complexity is reduced by eliminating the need for maintaining a dedicated custom backend infrastructure. In addition to Firebase, the system integrates external third-party services (e.g., weather, map, and satellite APIs), which are accessed via RESTful APIs over

HTTPS.
This architectural approach emphasizes a client-driven design, reduces backend complexity, and enables scalability by leveraging managed cloud services.

This approach removes the need for a dedicated backend gateway or custom server-side components, allowing the application to directly orchestrate interactions with backend services and external resources.

2.2 Component view

2.2.1 High level component view

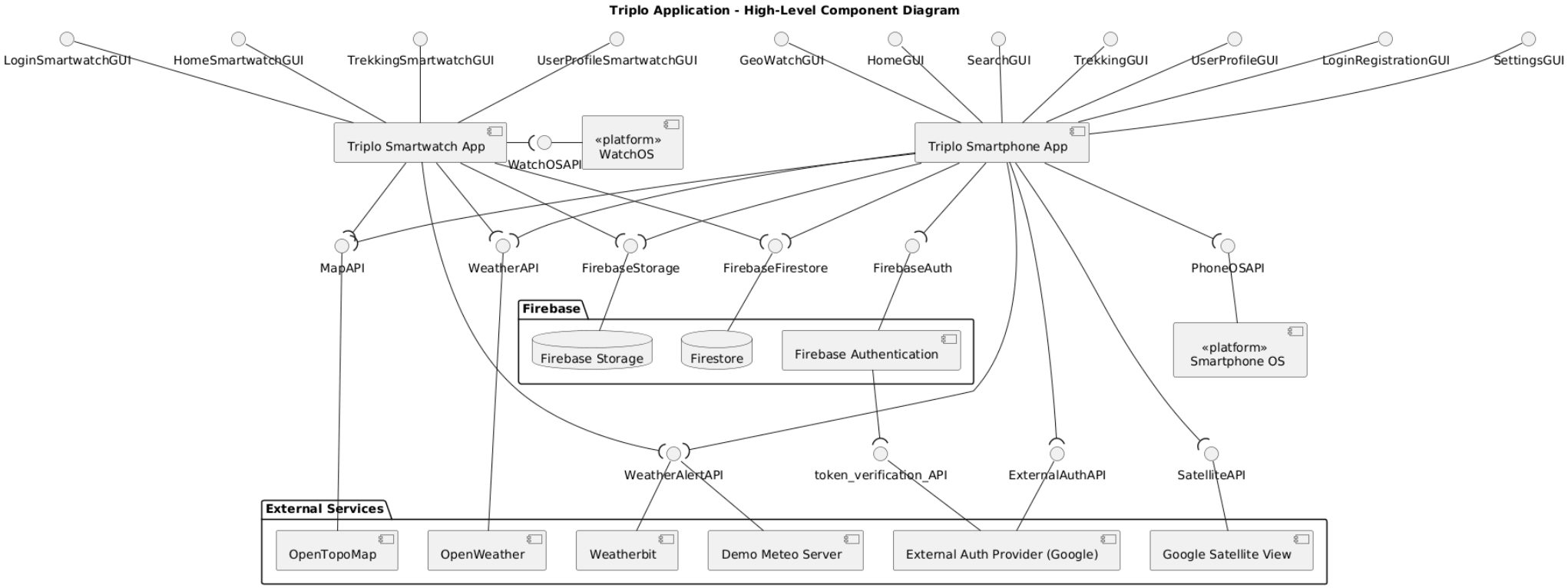


Figure 2.3: High level component view

The high-level component diagram provides an overview of the main architectural elements of the system and their interactions. The system is composed of two client applications, namely the smartphone application and the smartwatch application, both relying on a shared set of backend services and external resources.
The graphical user interfaces (GUIs) are represented following the convention adopted in both the *Software Engineering* and *UML Distilled* books quoted in the references. In this context, each GUI represents a group of related pages or a major functional area of the application, rather than a single page.
The diagram highlights the presence of two versions of the application. The smartphone application represents the main client and provides the full set of functionalities. It is implemented using Flutter, which enables cross-platform development and allows the application to run on both Android and iOS devices, as explained in section 2.7.
The smartwatch application represents a lightweight extension of the system and is deployed on Wear OS, as discussed in section 2.7 . It provides a subset of functionalities adapted to the constraints of wearable devices.
Both applications interact with a common set of backend services and selected external resources. In particular, both the smartphone and smartwatch applications access Firestore for data storage and Firebase Storage for media management, as well as shared external services such as OpenTopoMap for map visualization. However, the smartwatch application is intentionally restricted to a subset of services, reflecting its role as a lightweight client and the constraints of wearable devices.
An architectural difference regards authentication. While the smartphone application directly relies on Firebase Authentication, the smartwatch application does not perform authentication independently. Instead, it adopts a pairing mechanism based on QR code scanning. Through this process, the smartphone securely associates an authenticated user with the smartwatch by storing the corresponding identifier in Firestore.
This design choice provides several advantages, detailed in section 2.7.3

Access to backend resources such as Firestore, Firebase Storage, and selected external services is otherwise shared between the smartphone and smartwatch applications. The diagram explicitly shows which services are accessible from each client, clarifying the distribution of responsibilities.
The system also integrates multiple external services, for example map providers such as OpenTopoMap and Google Satellite View, as well as external authentication providers. Weather alert functionalities rely on both Weatherbit and a Demo Meteo Server. The latter is used exclusively for demonstration purposes, allowing the simulation of weather alerts without depending on real external conditions.

The operating system interfaces are modeled as platform components, Smartphone OS and Watch OS. These platforms are not accessed directly by the application; instead, access is mediated through Flutter packages that provide abstractions over native operating system services. The specific packages used to interact with such services are detailed in the diagrams of the smartphone application components, section 2.2.2 and of the smartwatch application components, section 2.2.3.
The interfaces associated with Firebase services, Firebase Authentication, Firestore, and Firebase Storage are based on the SDKs provided by the Backend-as-a-Service platform. These SDKs abstract the underlying communication mechanisms and offer a structured way to access backend functionalities, as documented in the Firebase official documentation.
In contrast, the remaining external interfaces, such as Weather API, Weather Alert API, Satellite API, are accessed through RESTful communication. In these cases, the application uses HTTP-based requests via Flutter libraries, handling data exchange in JSON format and performing client-side decoding.
The token verification API is not directly managed within the application logic but is represented the complete authentication flow. In particular, when authentication is performed through an external provider, Google, the application obtains an authentication token which is then passed to Firebase Authentication. Firebase subsequently validates this token with the external provider as part of the OAuth2-based authentication process. This additional interaction is abstracted by the SDK but is included in the diagram to provide a more accurate representation of the overall system behavior.

2.2.2 Mobile Smartphone App component view

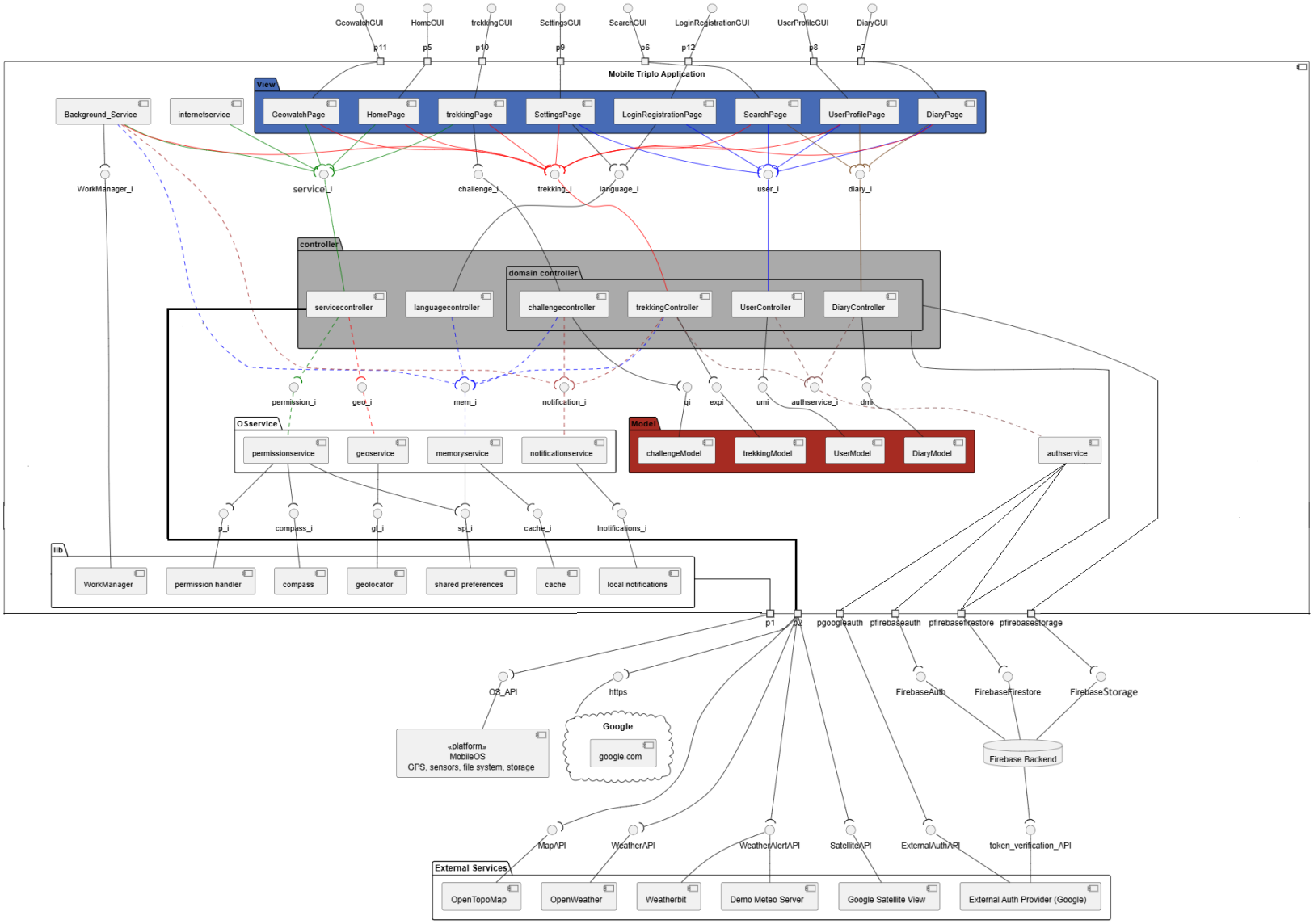


Figure 2.4: Smartphone Application component view

View Components

The UI of the application is organized into components that group together multiple views related to the same functionality, rather than representing isolated pages. This choice reflects how users interact with the system, as each View component encapsulates a complete feature.

The `GeoWatch Page component` includes all functionalities related to navigation and environmental awareness during a trekking activity. It is responsible for presenting weather information along the route, weather alerts, and a satellite map with meteorological layers. It also provides navigation support through compass heading, altitude, and user position.

The `Home Page component` represents the main map-based point of the application. It displays trekking routes on an OpenTopoMap layer and allows users to explore available paths geographically.

The `Trekking Page component` provides detailed information about a selected route, including a description and photos. It also allows users to access the component that handles the creation of a diary associated with the route and to save the route to their profile, the `Diary Page component`.

The `Diary Page component` handles the creation and modification of diary entries related to trekking experiences, including notes, images, and activity-related data.

The `User Profile Page component` is responsible for displaying user information, including saved trekkings, diary related, follower and following users' data.

The `Settings Page component` allows users to manage application and account settings, including profile information, security options such as password updates, smartwatch pairing, and weather notification preferences.

The `LoginRegistration Page component` manages login and registration workflows. It supports authentication via email and password, performs simple format validation operations, supports Google authentication, and password recovery.

The `Search Page component` enables users to search for other users and access their public profiles or to search for a trekking by its name.

Controller components

The controller package coordinates the application logic and mediates interactions between UI components, models, and backend services.

Domain controllers, such as `UserController`, `DiaryController`, `TrekkingController`, and `ChallengeController`, are responsible for handling use cases specific to their domain. They orchestrate data flow between the UI and Firebase services.

The `ServiceController` acts as a unified facade for functionalities that do not belong to an application domain defined in the model, such as access to external APIs and operating system services, reducing coupling between UI components and low-level implementations.

The `LanguageController` is responsible for managing localization and language-related logic.

OSService Package components

The OSService package contains services that directly interact with Flutter libraries responsible to communicate with system APIs and services managed by the operating system.

These include components such as `GeoService`, `PermissionService`, `MemoryService`, and `NotificationSe` each encapsulating a specific interaction with the operating system, such as location access, permission handling, local storage, and notifications.

This package isolates dependencies on external libraries and platform APIs. If a library needs to be replaced or updated, the changes are confined to the corresponding service, without affecting the rest of the application.

Internet Service

The `InternetService` is responsible for monitoring network connectivity. It periodically relies on the ServiceController to attempt connections to external endpoints (e.g., Google services). Based on the outcome, the application switches between offline and online modes, ensuring consistent behavior in varying network conditions.

Background Service

The `BackgroundService` enables the execution of tasks even when the application is not actively open, depending on operating system constraints.

It is used to perform periodic operations such as checking weather conditions and generating notifications. These tasks are scheduled through the mechanisms WorkManager mechanism, a Flutter wrapper around Android's Workmanager and iOS Background tasks. Background tasks are executed based on system availability.

AuthService

The AuthService is responsible not only for authentication but also for managing the smartwatch pairing process.

It handles user authentication through Firebase Auth and Google, password reset, session management, coordinates the pairing workflow between the smartphone application and the smartwatch, centralizing all identity-related logic.

Backend and External Services

The application relies on Firebase as a Backend-as-a-Service, including Authentication, Firestore, and Storage.

External services are used for specific functionalities, such as OpenTopoMap for map visualization, OpenWeather and Weatherbit for weather data and alerts, and Google services for authentication and connectivity verification.

Architectural Summary

The architecture is organized in packages that separate concerns clearly. Domain controllers manage application logic and backend interaction, the ServiceController provides a unified access point to external and system services, and OSService components encapsulate platform dependencies. This structure improves modularity, maintainability, and adaptability of the system.

Domain controllers, such as `UserController`, `DiaryController`, `TrekkingController`, and `ChallengeCon` are responsible for coordinating the application logic related to their respective functional domains. These controllers manage interactions between the View layer and the Model layer, and they also mediate access to backend resources such as Firebase services.

Therefore, in the proposed architecture:

- domain controllers manage the application use cases and the interaction with Firebase-related resources;
- The ServiceController acts as a unified facade aggregating multiple underlying services, including external APIs (weather, maps) and device-level services (location, permissions, navigation)
- Firebase is modeled as a set of backend services used by the application, rather than as a custom backend.

Smartphone Application External Libraries

Package	Purpose
flutter	Core UI toolkit for building cross-platform applications
provider	State management and implementation of the Observer pattern
firebase_core	Initialization of Firebase services
firebase_auth	User authentication management
cloud_firestore	Firebase cloud database for application data
firebase_storage	Cloud storage for images and media files
google_sign_in	Google authentication integration
flutter_map	Map rendering using OpenStreetMap tiles
latlong2	Geographic coordinate utilities
geolocator	Device location services
geocoding	Address and location conversion
flutter_compass	Compass and orientation data
image_picker	Image selection from device gallery or camera
mobile_scanner	QR code scanning functionality
shared_preferences	Local persistent key-value storage
flutter_local_notifications	Local notification management
workmanager	Background tasks
permission_handler	Runtime permission handling
flutter_dotenv	Environment variable management
uuid	Unique identifiers
intl	Internationalization and localization utilities
share_plus	Content sharing across apps
flutter_cache_manager	Image and file caching
cupertino_icons	iOS-style icon set

Table 2.1: Main dependencies used in the smartphone application

2.2.3 Mobile Smartwatch App component view

2.3 Deployment view

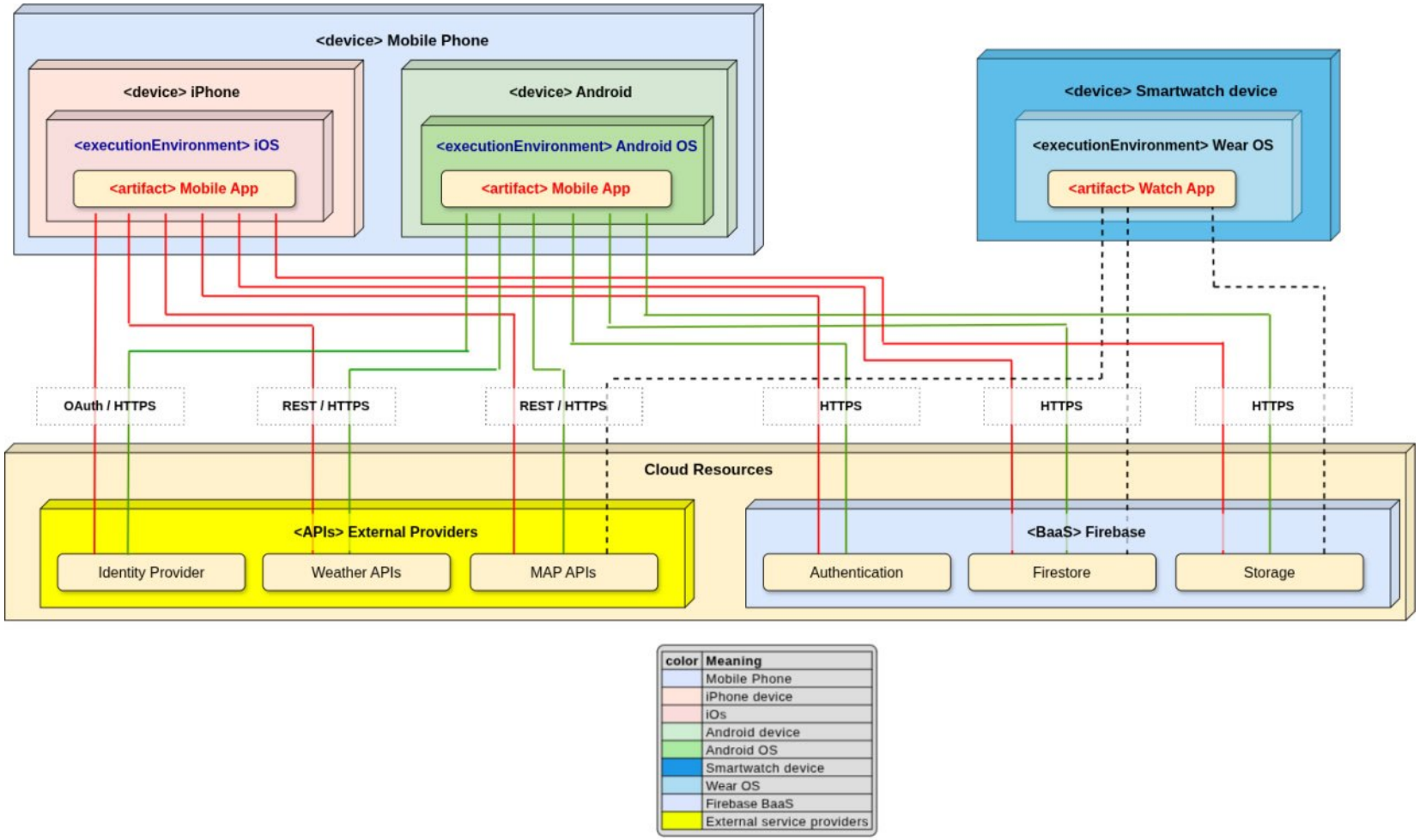


Figure 2.5: Deplyment Diagram

The deployment diagram is consistent with the UML deployment notation described in chapther 8 of the UML Distilled book and on page 52 of the Software Engineering book listed in the references. Physical hardware units are modeled through the `«device»` stereotype, while software platforms and runtime contexts are represented through the `«executionEnvironment»` stereotype.

The deployed software elements are shown as `«artifact»`. Communication paths are represented by links annotated with the corresponding protocol or interaction type, such as HTTPS, REST/HTTPS, and OAuth/HTTPS.

In particular, the diagram distinguishes the mobile deployment on Android and iOS devices, while the smartwatch deployment is limited to Wear OS. The cloud side is modeled as a device containing different execution environments, namely Firebase and the set of external providers, which host the artifacts used by the applications.

The deployment diagram represents the distribution of the application artifacts across different execution environments. In particular, the smartphone application is deployed as an artifact on both iOS and Android execution environments, while the smartwatch application is deployed separately on a Wear OS execution environment.

Although the smartphone application is shown as deployed on two distinct platforms, it is important to note that it is based on a single shared codebase. The application is developed using Flutter, a cross-platform framework that allows the same source code to be compiled and executed on both iOS and Android devices. As a result, the two deployments correspond to different execution environments of the same application, rather than distinct implementations. In contrast, the smartwatch application is implemented as a separate artifact. While it shares the same backend services and part of the architectural structure, it is specifically designed for the Wear OS environment and provides a reduced set of functionalities for wearable devices. Therefore, it is represented as a distinct deployment unit in the diagram.

2.4 Runtime view

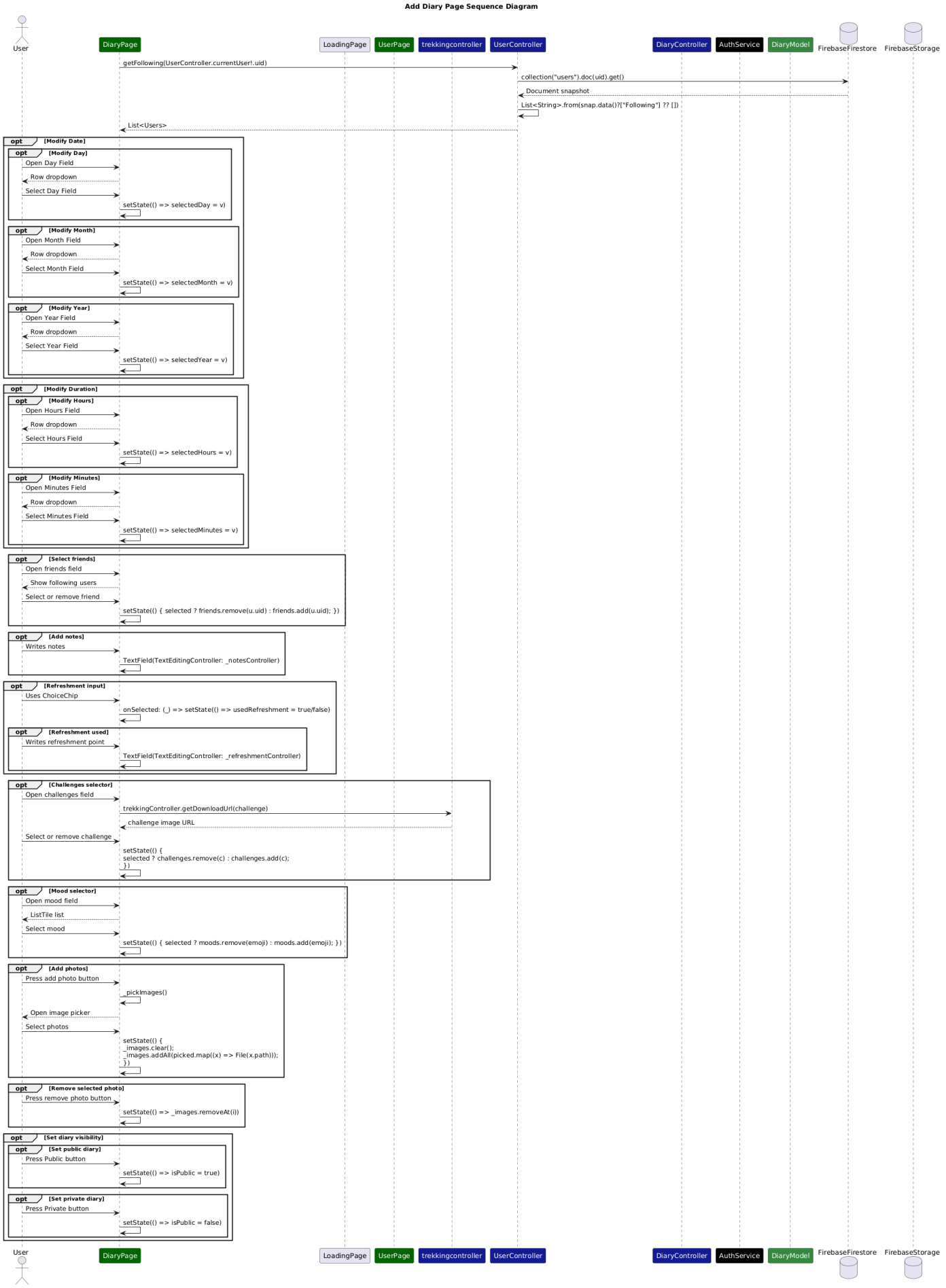
2.4.1 Selected Use Cases

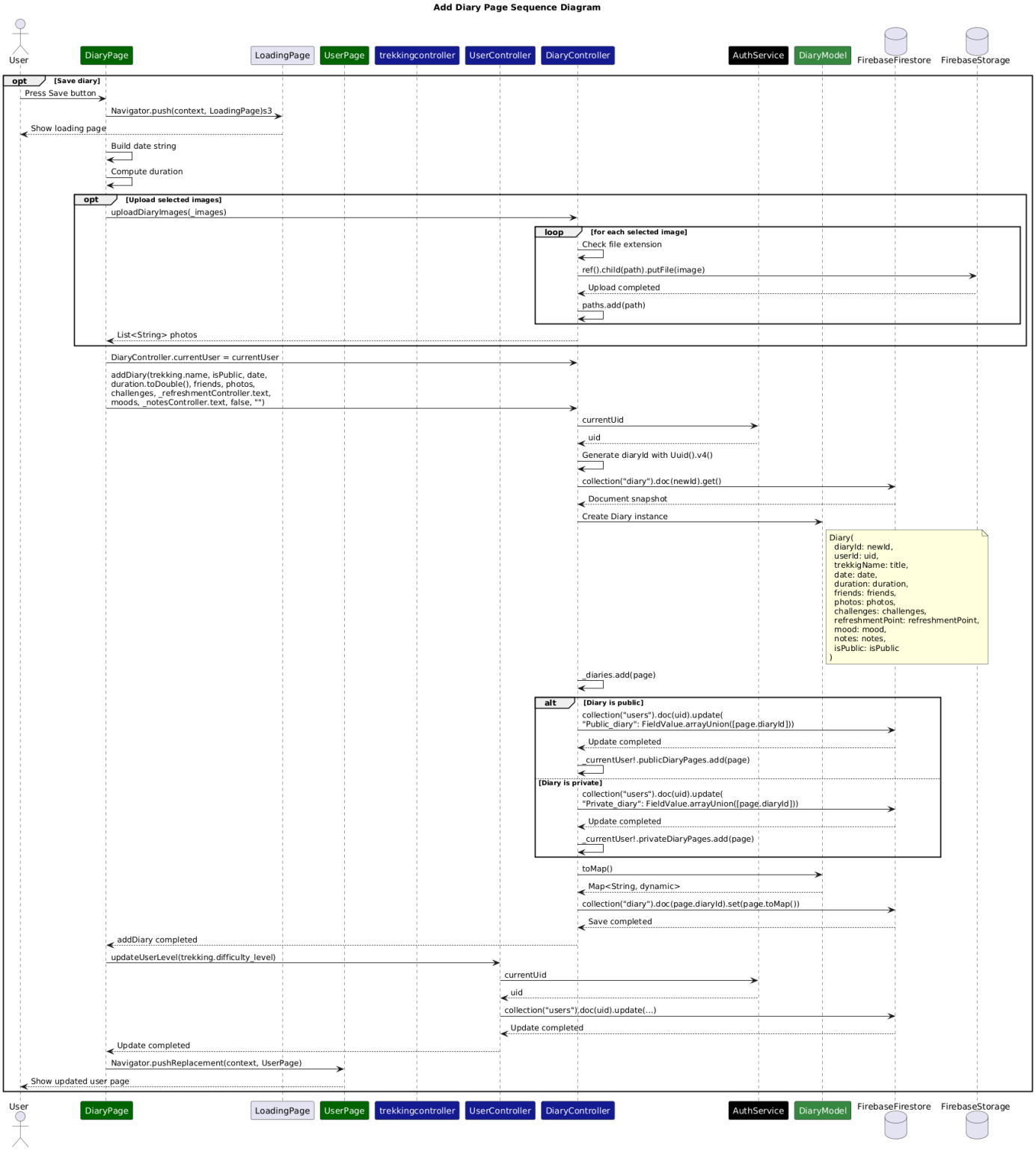
The identified use cases provide an abstract representation of the interactions between the user and the system. They have been selected to capture the essential functionalities of the application and to model its observable behavior from the user's perspective.
These set of use cases is representative of the main system capabilities and supports a clear understanding of the services offered to the user.

- Add Diary Page
- Explore trekkings on Map
- View trekking information
- View weather information
- Search Users
- Search trekking
- View compass and altitude information
- Login
- Register
- View User Information
- Pair Smartwatch
- Update User Profile

2.4.2 Add Diary Page use case sequence diagram

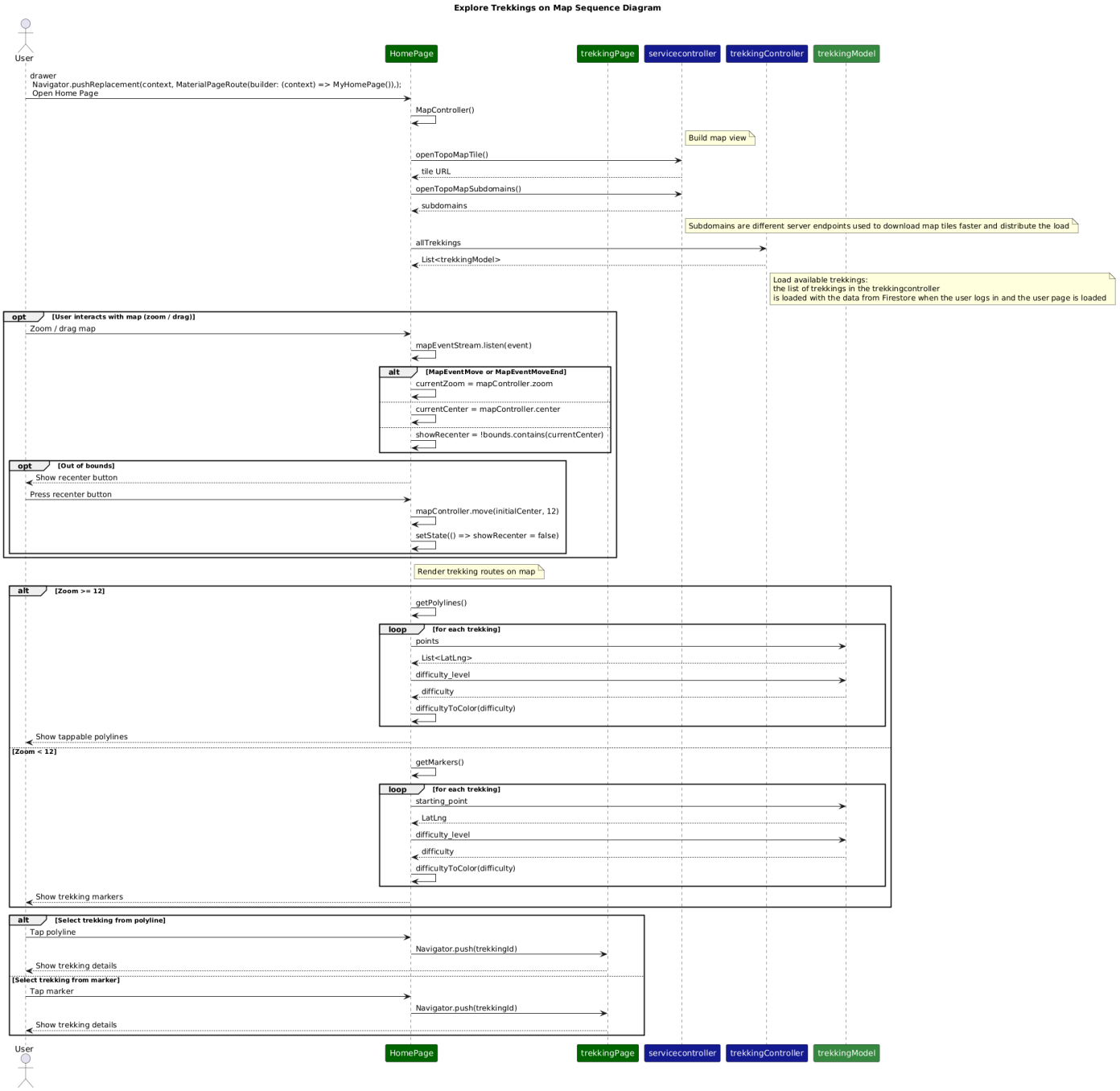
The most important part of the *Add Diary Page* sequence diagram is the save phase, where the data entered by the user is stored in the system. When the user presses the save button, the `DiaryPage` starts the process by showing a loading page and then it formats the date, computing the duration from hours and minutes. If the user has selected photos, the page calls the `DiaryController` to upload them to `FirebaseStorage`. The controller uploads each image and returns a list of paths, which will be stored in the diary instance. After this step, the `DiaryPage` calls the `addDiary()` method of the `DiaryController`, passing all the collected data. The controller retrieves the current user identifier from `FirebaseAuth`, generates a new unique identifier for the diary, and creates a `DiaryModel` object. Before saving the diary in `FirebaseFirestore`, the object is converted into a `Map<String, dynamic>`. This is necessary because Firestore stores data in a JSON-like format, which is based on key-value pairs. The map represents the diary as a set of fields (such as date, duration, notes, etc.), where each field is associated with a value. The type `dynamic` is used because these values can have different types, such as strings, numbers, booleans, or lists. Once converted, the map is stored in the `diary` collection in Firestore. At the same time, the controller updates the user document, adding the diary identifier to either the public or private diary list, depending on the selected visibility. Finally, the control returns to the `DiaryPage`, which updates the user level through the `UserController` and navigates to the `UserPage`, completing the operation.


Add Diary Page Sequence Diagram
User
DiaryPage
LoadingPage
UserPage
trekkingcontroller
UserController
DiaryController
AuthService
DiaryModel
FirebaseFirestore
FirebaseStorage
getFollowing(UserController.currentUser!.uid)
collection("users").doc(uid).get()
Document snapshot
List<String>.from(snap.data()?["Following"] ?? [])
List<Users>
opt [Modify Date]
opt [Modify Day]
Open Day Field
Row dropdown
Select Day Field
setState(() => selectedDay = v)
opt [Modify Month]
Open Month Field
Row dropdown
Select Month Field
setState(() => selectedMonth = v)
opt [Modify Year]
Open Year Field
Row dropdown
Select Year Field
setState(() => selectedYear = v)
opt [Modify Duration]
opt [Modify Hours]
Open Hours Field
Row dropdown
Select Hours Field
setState(() => selectedHours = v)
opt [Modify Minutes]
Open Minutes Field
Row dropdown
Select Minutes Field
setState(() => selectedMinutes = v)
opt [Select friends]
Open friends field
Show following users
Select or remove friend
setState(() { selected ? friends.remove(u.uid) : friends.add(u.uid); })
opt [Add notes]
Writes notes
TextField(TextEditingController: _notesController)
opt [Refreshment input]
Uses ChoiceChip
onSelected: (_) => setState(() => usedRefreshment = true/false)
opt [Refreshment used]
Writes refreshment point
TextField(TextEditingController: _refreshmentController)
opt [Challenges selector]
Open challenges field
trekkingController.getDownloadUrl(challenge)
challenge image URL
Select or remove challenge
setState(() {
selected ? challenges.remove(c) : challenges.add(c);
})
opt [Mood selector]
Open mood field
ListTile list
Select mood
setState(() { selected ? moods.remove(emoji) : moods.add(emoji); })
opt [Add photos]
Press add photo button
_pickImages()
Open image picker
Select photos
setState(() {
_images.clear();
_images.addAll(picked.map((x) => File(x.path)));
})
opt [Remove selected photo]
Press remove photo button
setState(() => _images.removeAt(i))
opt [Set diary visibility]
opt [Set public diary]
Press Public button
setState(() => isPublic = true)
opt [Set private diary]
Press Private button
setState(() => isPublic = false)


Add Diary Page Sequence Diagram
User
DiaryPage
LoadingPage
UserPage
trekkingcontroller
UserController
DiaryController
AuthService
DiaryModel
FirebaseFirestore
FirebaseStorage
opt [Save diary]
Press Save button
Navigator.push(context, LoadingPage)s3
Show loading page
Build date string
Compute duration
opt [Upload selected images]
uploadDiaryImages(_images)
loop [for each selected image]
Check file extension
ref().child(path).putFile(image)
Upload completed
paths.add(path)
List<String> photos
DiaryController.currentUser = currentUser
addDiary(trekking.name, isPublic, date, duration.toDouble(), friends, photos, challenges, _refreshmentController.text, moods, _notesController.text, false, "")
currentUid
uid
Generate diaryId with Uuid().v4()
collection("diary").doc(newId).get()
Document snapshot
Create Diary instance
Diary(
diaryId: newId,
userId: uid,
trekkigName: title,
date: date,
duration: duration,
friends: friends,
photos: photos,
challenges: challenges,
refreshmentPoint: refreshmentPoint,
mood: mood,
notes: notes,
isPublic: isPublic
)
_diaries.add(page)
alt [Diary is public]
collection("users").doc(uid).update("Public_diary": FieldValue.arrayUnion([page.diaryId]))
Update completed
_currentUser!.publicDiaryPages.add(page)
[Diary is private]
collection("users").doc(uid).update("Private_diary": FieldValue.arrayUnion([page.diaryId]))
Update completed
_currentUser!.privateDiaryPages.add(page)
toMap()
Map<String, dynamic>
collection("diary").doc(page.diaryId).set(page.toMap())
Save completed
addDiary completed
updateUserLevel(trekking.difficulty_level)
currentUid
uid
collection("users").doc(uid).update(...)
Update completed
Update completed
Navigator.pushReplacement(context, UserPage)
Show updated user page

2.4.3 Explore Trekkings on Map Sequence Diagram

The sequence diagram for the *Explore Trekkings on Map* use case describes how the user interacts with the main map interface in order to visualize the available trekking routes. The interaction starts when the user opens the `HomePage`, which initializes the map component and requests the information needed to render the map background through the `servicecontroller`. In particular, the page retrieves the tile URL template and the corresponding subdomains used by the map provider. Once the map has been initialized, the `HomePage` accesses the trekking data exposed by the `trekkingController`. The available trekkings are then rendered differently depending on the zoom level and the rendering logic uses the trekking information, such as route points, starting point, and difficulty level, in order to determine the visual representation. The diagram also highlights the dynamic behavior of the map during user interaction. When the user drags or zooms the map, the page updates the current zoom and center, and checks whether the visible area is still inside the bounds. If the user moves outside the expected area, a recenter button is shown. Finally, when the user selects a trekking, the system navigates from the `HomePage` to the `trekkingPage`, where the detailed trekking information is displayed.



2.4.4 View Trekking Information Sequence Diagram

The sequence diagram for the *View Trekking Information* use case describes how the system presents the details of a selected trekking. The interaction starts when the user opens the `trekkingPage`. The page initializes its internal state and verifies which user is authenticated through the `UserController`. The trekking data is then retrieved through the `trekkingController`. In particular, the controller attempts to search in its local collection. After the trekking has been obtained, the page checks whether the route is already saved by the current user. The page also requests weather information near the trail through the `servicecontroller`. A further part of the interaction regards the trekking images, such as the map photo, the ending point photo, and the challenge images. In the main sequence diagram, this step is represented at a high level through calls from the `trekkingPage` to the `trekkingController`. The detailed behavior of the cache mechanism used for image retrieval is described separately in a separate diagram, in order to keep the main diagram focused on the overall interaction flow. Finally, the page allows the user to perform additional actions related to the trekking. The route can be saved or removed from the saved list, a diary entry can be created through the `DiaryPage`, and the user can navigate to other trekking-related views such as the route details page or the geowatch and map page. In this way, the `trekkingPage` acts as the central entry point for the main information and actions associated with a trekking.

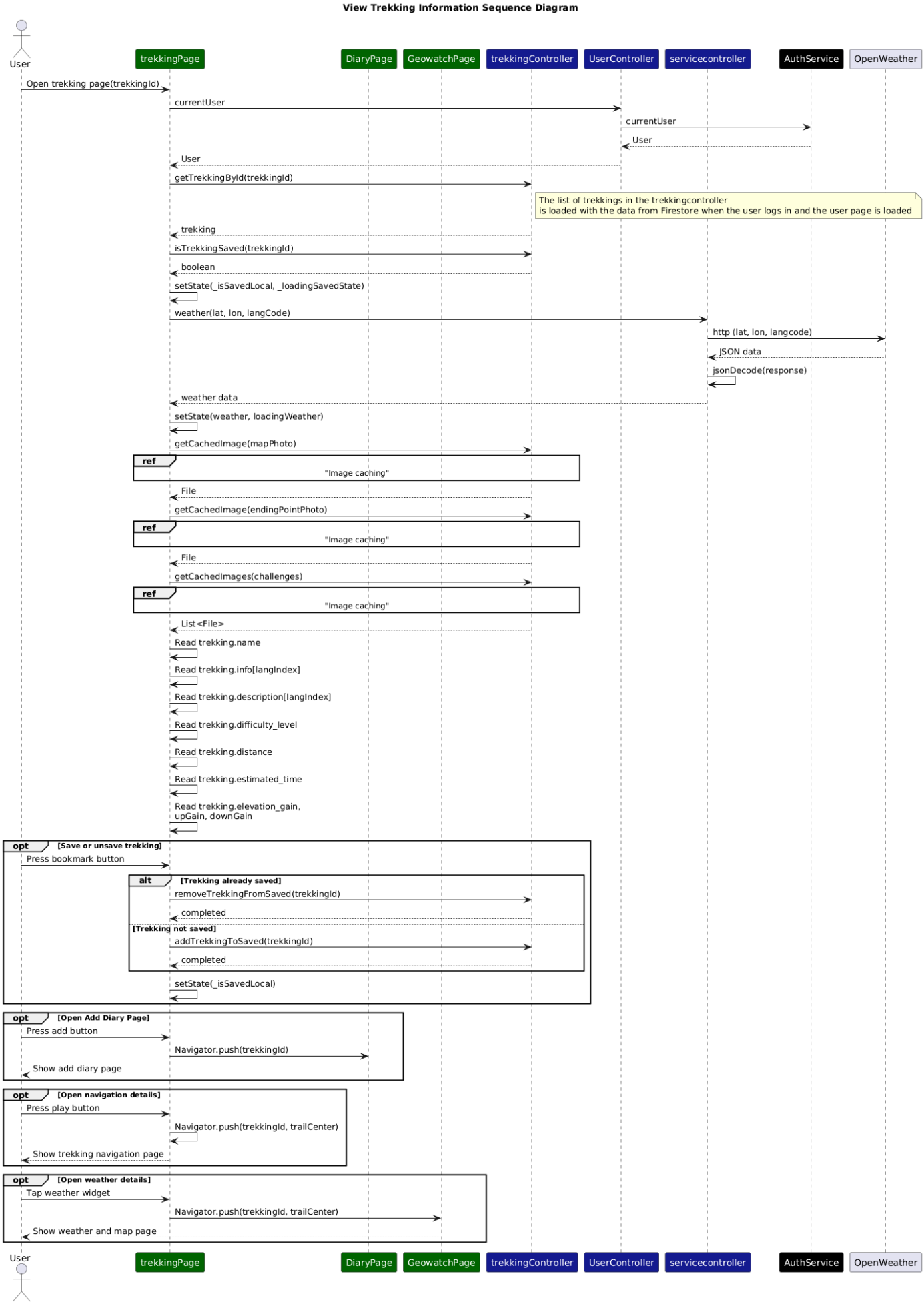

View Trekking Information Sequence Diagram
User
trekkingPage
DiaryPage
GeowatchPage
trekkingController
UserController
servicecontroller
AuthService
OpenWeather
Open trekking page(trekkingId)
currentUser
currentUser
User
User
getTrekkingById(trekkingId)
The list of trekkings in the trekkingcontroller
is loaded with the data from Firestore when the user logs in and the user page is loaded
trekking
isTrekkingSaved(trekkingId)
boolean
setState(_isSavedLocal, _loadingSavedState)
weather(lat, lon, langCode)
http (lat, lon, langcode)
JSON data
jsonDecode(response)
weather data
setState(weather, loadingWeather)
getCachedImage(mapPhoto)
ref
"Image caching"
File
getCachedImage(endingPointPhoto)
ref
"Image caching"
File
getCachedImages(challenges)
ref
"Image caching"
List<File>
Read trekking.name
Read trekking.info[langIndex]
Read trekking.description[langIndex]
Read trekking.difficulty_level
Read trekking.distance
Read trekking.estimated_time
Read trekking.elevation_gain,
upGain, downGain
opt
[Save or unsave trekking]
Press bookmark button
alt
[Trekking already saved]
removeTrekkingFromSaved(trekkingId)
completed
[Trekking not saved]
addTrekkingToSaved(trekkingId)
completed
setState(_isSavedLocal)
opt
[Open Add Diary Page]
Press add button
Navigator.push(trekkingId)
Show add diary page
opt
[Open navigation details]
Press play button
Navigator.push(trekkingId, trailCenter)
Show trekking navigation page
opt
[Open weather details]
Tap weather widget
Navigator.push(trekkingId, trailCenter)
Show weather and map page
User
trekkingPage
DiaryPage
GeowatchPage
trekkingController
UserController
servicecontroller
AuthService
OpenWeather

Image Caching for Trekking Images Sequence Diagram

This sequence diagram illustrates, through a representative example, how the image caching mechanism is implemented within the application. Its purpose is not to introduce a new use case, but rather to clarify the internal behavior of image retrieval, which is abstracted in the main sequence diagram describing the trekking visualization flow. The process starts when the `trekkingPage` requests an image through the `trekkingController`. The controller delegates the retrieval to the `memoryservice`, which implements a multi-level caching strategy. First, the system attempts to retrieve the image from the in-memory RAM cache, using a `Map<String, File>` structure. If the image is found, it is immediately returned to the page, ensuring minimal latency. If the image is not available in memory, the system checks the disk cache through the `CacheManager`. This involves interacting with the underlying operating system to load a previously stored file. If the image is found on disk, it is reinserted into the in-memory cache and returned to the UI, improving performance for subsequent accesses. If the image is not available in either RAM or disk cache, the system performs a download from `FirebaseStorage`. In the case of paths expressed using the `gs://` protocol, the controller first resolves them into a downloadable URL. The image is then downloaded, stored on disk, and finally cached in memory before being returned to the page. The same logic is applied in a batched manner when multiple images, for example trekking challenges, are requested. In this case, the controller iterates over all image paths and applies the same caching strategy to each element.

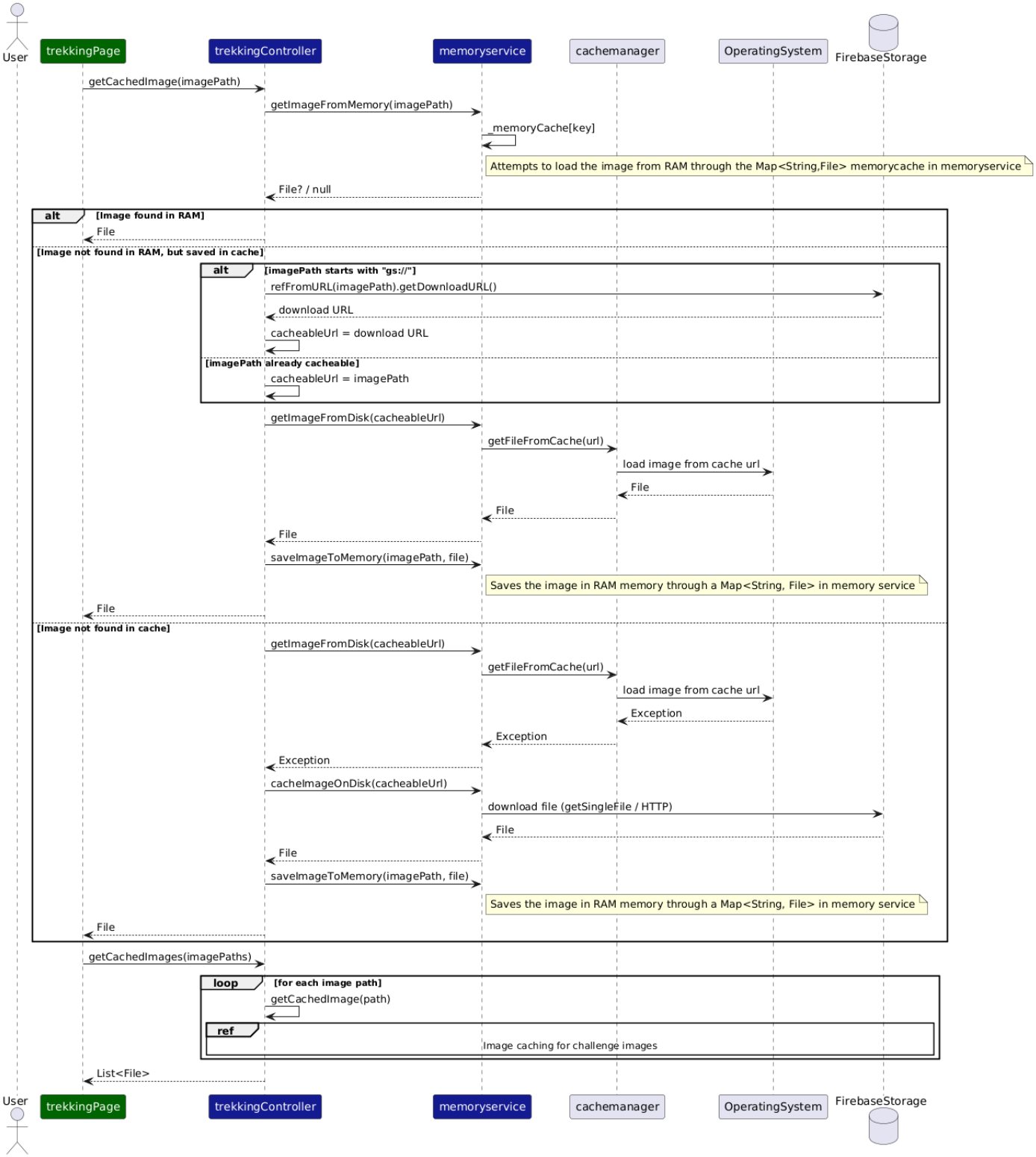


Image caching for trekking images sequence diagram
User
trekkingPage
trekkingController
memoryservice
cachemanager
OperatingSystem
FirebaseStorage
getCachedImage(imagePath)
getImageFromMemory(imagePath)
_memoryCache[key]
Attempts to load the image from RAM through the Map<String,File> memorycache in memoryservice
File? / null
alt [Image found in RAM]
File
[Image not found in RAM, but saved in cache]
alt [imagePath starts with "gs://"]
refFromURL(imagePath).getDownloadURL()
download URL
cacheableUrl = download URL
[imagePath already cacheable]
cacheableUrl = imagePath
getImageFromDisk(cacheableUrl)
getFileFromCache(url)
load image from cache url
File
File
File
saveImageToMemory(imagePath, file)
Saves the image in RAM memory through a Map<String, File> in memory service
File
[Image not found in cache]
getImageFromDisk(cacheableUrl)
getFileFromCache(url)
load image from cache url
Exception
Exception
Exception
cacheImageOnDisk(cacheableUrl)
download file (getSingleFile / HTTP)
File
File
saveImageToMemory(imagePath, file)
Saves the image in RAM memory through a Map<String, File> in memory service
File
getCachedImages(imagePaths)
loop [for each image path]
getCachedImage(path)
ref
Image caching for challenge images
List<File>

2.4.5 View Weather Information Sequence Diagram

The `Geowatch` component is responsible for retrieving, processing, and displaying weather information related to a trekking route or the user's current position. This functionality is implemented through the `ServiceController`, which acts as a facade for external APIs. The main logic is handled by the method `_loadAll()`, which is responsible for orchestrating the full weather retrieval process. This method:

- determines the target location (trekking route or user GPS position),
- retrieves current weather data,
- retrieves forecast data,
- parses the forecast into a simplified structure,
- retrieves weather alerts.

The method is invoked:

- once during initialization,
- periodically through a timer (every 5 minutes),
- when the user switches between trail-based and GPS-based weather.

The method `_loadAlertState()` is responsible for retrieving the current notification status for the selected trekking route. It queries the `TrekkingController`, which:

- retrieves the current user identifier through `AuthService`,
- queries Firestore to check whether the trekking is present in the user's alert list.

The result determines the state of the notification icon in the UI. The method `weather(lat, lon, lang)` performs an HTTP request to the OpenWeather API. The JSON response is decoded into a `Map<String, dynamic>` and sent over to the UI. The method `forecast(lat, lon, lang)` retrieves raw forecast data from OpenWeather. Since the response contains multiple time-based entries, a parsing phase is required. This parsing is handled by the following methods in the `ServiceController`:

- `parseForecastItem(raw)`: converts a single raw JSON entry into a simplified structure (date, temperature, icon, description),
- `parseForecast(rawList)`: applies the transformation to all entries.

Weather alerts are obtained from two sources:

- `weatherbitAlerts(lat, lon)` for real-time alerts,
- `mockAlerts()` for testing purposes.

The results are merged within the UI layer to provide a unified alert visualization. The `GoogleSatellitePag` part of the `Geowatch` component, provides an alternative map visualization using satellite imagery. The page retrieves tile URLs through the `ServiceController` and optionally overlays weather layers, such as pressure, rain and others using OpenWeather tile services. Weather data is handled using `Map<String, dynamic>` structures. This allows flexible interaction with JSON responses, avoiding the need for rigid model definitions.

Handling Asynchronous Updates in Dynamic Pages The `mounted` property indicates whether the current page is still visible and active. When a page is removed (for example, after navigating to another screen), it is no longer safe to update its UI.

Since weather data is loaded asynchronously, it may happen that:

- a request is started,
- the user leaves the page,
- the request completes afterward.

If `setState` is called after the page has been removed, the application would crash.

For this reason, the code checks `if (!mounted) return;` before updating the UI, ensuring that updates are performed only when the page is still active.

Google Satellite integration The Google Satellite page does not directly perform HTTP requests to OpenWeather. Instead, it delegates the construction of the tile URL to the ServiceController.

The base map is not a single image but is composed of multiple small image tiles. The ServiceController provides a URL template for the map provider, for example Google Satellite for Geowatch or OpenTopoMap for the HomePage.

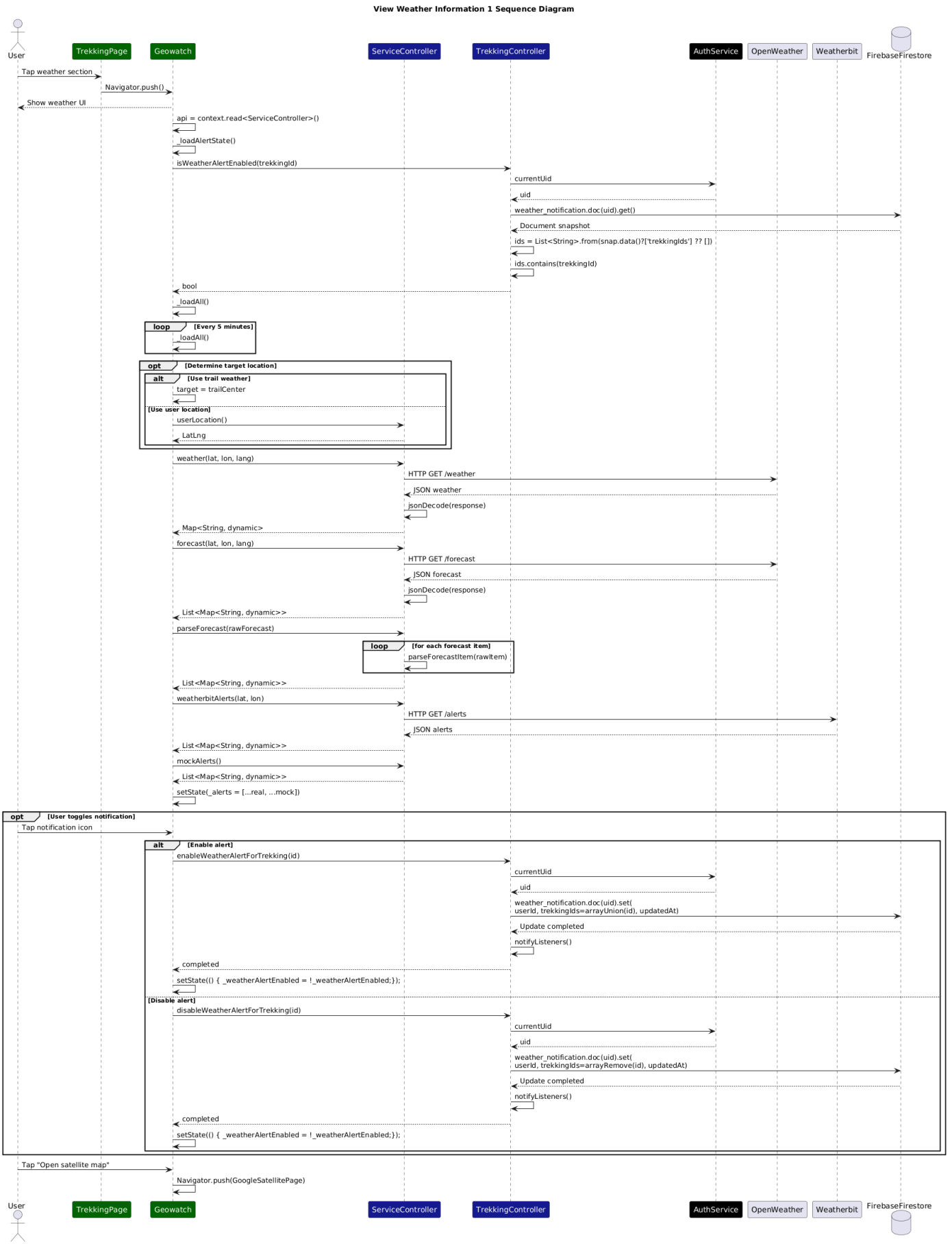
This URL contains placeholders such as `{z}`, `{x}`, and `{y}`, which represent:

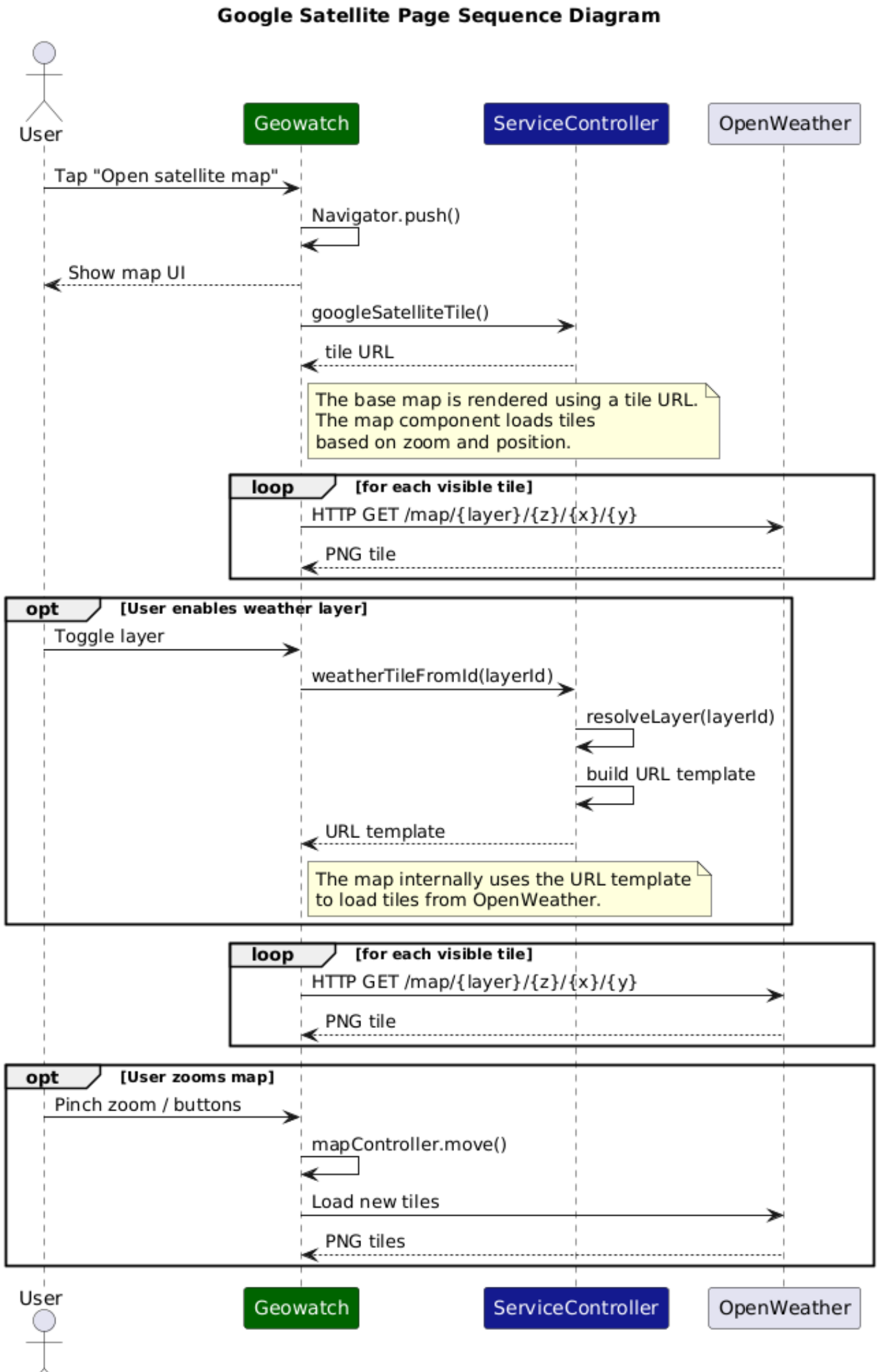
- the zoom level (`z`)
- the horizontal tile index (`x`)
- the vertical tile index (`y`)

The map component replaces these placeholders and loads only the tiles needed for the current view. When the user moves or zooms the map, new tiles are requested dynamically.

Architectural Separation: UI and API Responsibilities The UI does not directly perform HTTP requests for weather data. Instead, it delegates all external API interactions to the ServiceController.

An exception is the map rendering system, for both this map and the map in the home page, sequence diagram 2.4.3: once a tile URL template is provided, the map component internally handles tile requests to external providers. This behavior is part of the rendering library FlutterMap and does not violate the architectural separation, as the application logic does not manage these requests.


View Weather Information 1 Sequence Diagram
User
TrekkingPage
Geowatch
ServiceController
TrekkingController
AuthService
OpenWeather
Weatherbit
FirebaseFirestore
Tap weather section
Navigator.push()
Show weather UI
api = context.read<ServiceController>()
_loadAlertState()
isWeatherAlertEnabled(trekkingId)
currentUid
uid
weather_notification.doc(uid).get()
Document snapshot
ids = List<String>.from(snap.data()?['trekkingIds'] ?? [])
ids.contains(trekkingId)
bool
_loadAll()
loop [Every 5 minutes]
_loadAll()
opt [Determine target location]
alt [Use trail weather]
target = trailCenter
[Use user location]
userLocation()
LatLng
weather(lat, lon, lang)
HTTP GET /weather
JSON weather
jsonDecode(response)
Map<String, dynamic>
forecast(lat, lon, lang)
HTTP GET /forecast
JSON forecast
jsonDecode(response)
List<Map<String, dynamic>>
parseForecast(rawForecast)
loop [for each forecast item]
_parseForecastItem(rawItem)
List<Map<String, dynamic>>
weatherbitAlerts(lat, lon)
HTTP GET /alerts
JSON alerts
List<Map<String, dynamic>>
mockAlerts()
List<Map<String, dynamic>>
setState(_alerts = [...real, ...mock])
opt [User toggles notification]
Tap notification icon
alt [Enable alert]
enableWeatherAlertForTrekking(id)
currentUid
uid
weather_notification.doc(uid).set(
userId, trekkingIds=arrayUnion(id), updatedAt)
Update completed
notifyListeners()
completed
setState(() { _weatherAlertEnabled = !_weatherAlertEnabled;});
[Disable alert]
disableWeatherAlertForTrekking(id)
currentUid
uid
weather_notification.doc(uid).set(
userId, trekkingIds=arrayRemove(id), updatedAt)
Update completed
notifyListeners()
completed
setState(() { _weatherAlertEnabled = !_weatherAlertEnabled;});
Tap "Open satellite map"
Navigator.push(GoogleSatellitePage)


Google Satellite Page Sequence Diagram
User
Geowatch
ServiceController
OpenWeather
Tap "Open satellite map"
Navigator.push()
Show map UI
googleSatelliteTile()
tile URL
The base map is rendered using a tile URL.
The map component loads tiles
based on zoom and position.
loop
[for each visible tile]
HTTP GET /map/{layer}/{z}/{x}/{y}
PNG tile
opt
[User enables weather layer]
Toggle layer
weatherTileFromId(layerId)
resolveLayer(layerId)
build URL template
URL template
The map internally uses the URL template
to load tiles from OpenWeather.
loop
[for each visible tile]
HTTP GET /map/{layer}/{z}/{x}/{y}
PNG tile
opt
[User zooms map]
Pinch zoom / buttons
mapController.move()
Load new tiles
PNG tiles
User
Geowatch
ServiceController
OpenWeather

2.4.6 Search Users Sequence Diagram

The SearchPage component is responsible for handling user search interactions. The UI elements belong to the SearchPage component and communicate with the UserController.

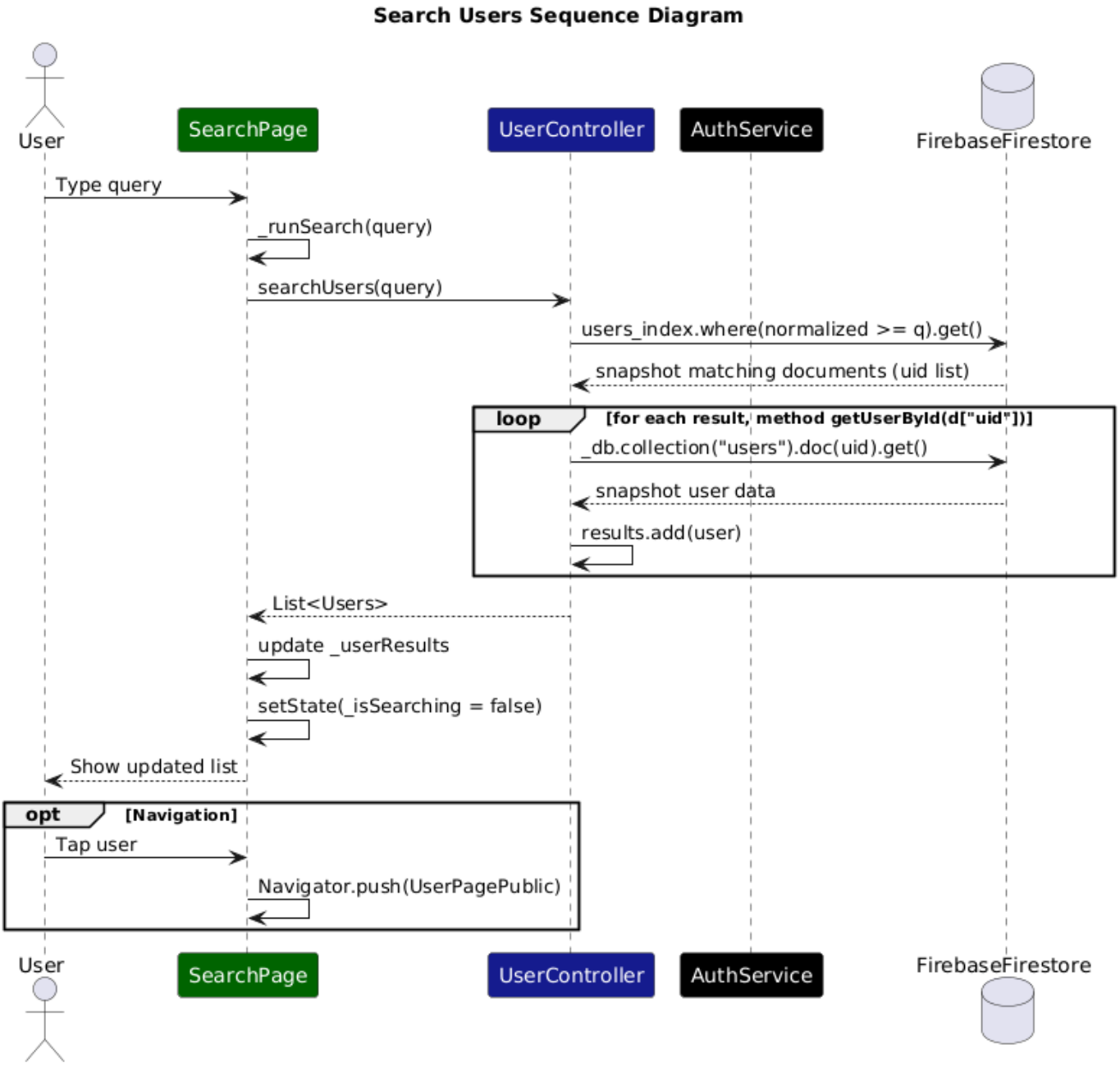
When the user types a query, the method `_runSearch(query)` is run by the `User controller`. Before performing the search, the UI state is updated using `setState` to indicate that a search operation is in progress.

The method `searchUsers(query)` is then invoked in the UserController.

The search process is performed in two steps:

- a query is executed on the `users_index` collection using a normalized version of the input string,
- for each result, the corresponding user document is retrieved from the `users` collection.

The results are returned as a list of `Users` objects. The SearchPage updates its internal state by assigning the results to `_userResults` and calling `setState`.



2.4.7 Search Trekking Sequence Diagram

The trekking search functionality follows a similar structure but introduces an additional optimization layer through local caching.

When the user enters a query, the method `_runSearch(query)` is executed, and the UI is updated using `setState` to reflect the loading state.

The request is delegated to the `searchTrekking(query)` method of the TrekkingController.

The search is performed in two phases:

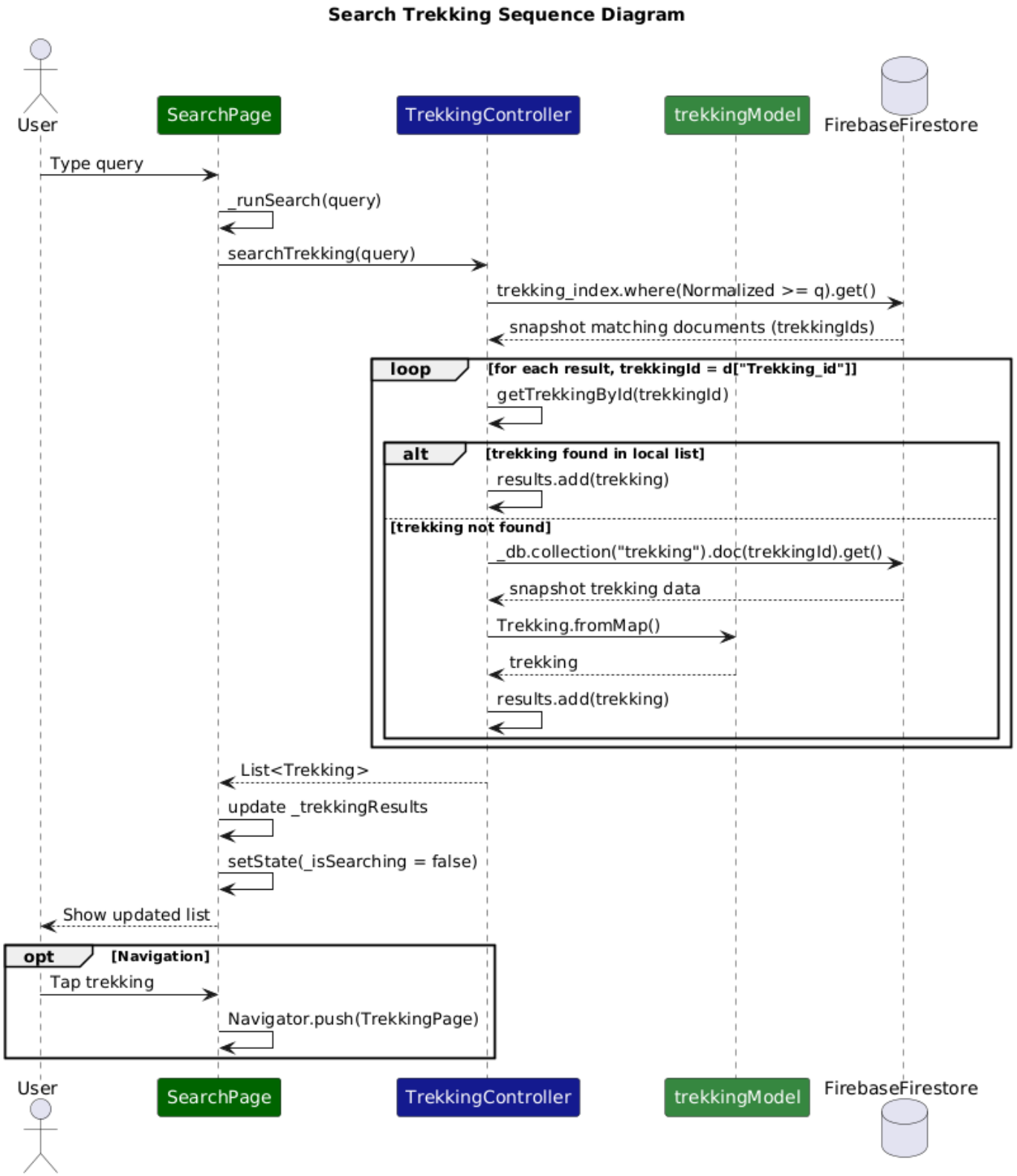
- the `trekking_index` collection is queried using a normalized string,
- each resulting trekking identifier is resolved into a full trekking object.

Before querying Firestore, the controller checks whether the trekking is already available in the local list. If the object is found locally, it is reused; otherwise, it is fetched from Firestore.

This approach reduces network overhead and improves performance.

Once the results are collected, they are returned to the SearchPage, which updates its internal state and rebuilds the UI using `setState`.

When the user selects a trekking, the application navigates to the corresponding TrekkingPage.


Search Trekking Sequence Diagram
User
SearchPage
TrekkingController
trekkingModel
FirebaseFirestore
Type query
_runSearch(query)
searchTrekking(query)
trekking_index.where(Normalized >= q).get()
snapshot matching documents (trekkingIds)
loop
[for each result, trekkingId = d["Trekking_id"]]
getTrekkingById(trekkingId)
alt
[trekking found in local list]
results.add(trekking)
[trekking not found]
_db.collection("trekking").doc(trekkingId).get()
snapshot trekking data
Trekking.fromMap()
trekking
results.add(trekking)
List<Trekking>
update _trekkingResults
setState(_isSearching = false)
Show updated list
opt
[Navigation]
Tap trekking
Navigator.push(TrekkingPage)
User
SearchPage
TrekkingController
trekkingModel
FirebaseFirestore

2.4.8 View Compass and Altitude Information Sequence Diagram

The `Geowatch` component is responsible for displaying compass, altitude, and current position information. This functionality is implemented through the interaction between the page, the `servicecontroller`, the `geoservice`, and the `permissionservice`.

When the navigation page is opened, the initialization starts in `initState()`, where the page registers itself as an app lifecycle observer and schedules the execution of `_initLocation()`. The observer is used to detect when the application returns to the foreground, for example after the user has changed GPS or permission settings.

The method `_initLocation()` delegates the preliminary checks to the `servicecontroller` through `loadNavigationLocation()`. This method verifies whether the location permission has been granted and whether the GPS service is enabled. If one of these conditions is not satisfied, the page enters an error state and shows a dedicated message to the user.

If permission is denied, the returned state contains the error `PERMISSION_DENIED`. If the GPS service is disabled, the returned state contains the error `GPS_DISABLED`. In both cases, the page updates its local state through `setState()` and displays an error instead of the navigation information.

Only when both checks are successful does the page subscribe to the position stream. The stream is obtained through the `servicecontroller`, which delegates the request to the `geoservice`. Every new `Position` object received from the stream is used to update altitude, latitude, longitude.

The compass information is handled through a separated stream. In the architecture, the UI does not access the `FlutterCompass` library. Instead, it requests the compass stream through the `servicecontroller`, which delegates the operation to the `geoservice`. The UI then receives heading values and converts them into cardinal directions through the helper method `_direction()`.

When an error state is shown, the page also provides a button that helps the user solve the problem. If the GPS is disabled, the button invokes `openGpsSettings()`, which is delegated through the `servicecontroller` to the `geoservice`. If the problem is related to permissions, the page instead invokes `openPermissionSettings()`, which is delegated to the `permissionservice`. In this way, the UI does not directly interact with operating system APIs.

The diagram also includes the handling of lifecycle changes. When the app returns to the foreground, it handles the app state change through method `didChangeAppLifecycleState()`. The page sets the checking state again and re-runs `_initLocation()`. This is useful, for example, when the user comes back from the system settings after enabling GPS or granting permissions.

Finally, when the page is closed, the observer and the active position stream subscription are disposed through the `dispose()` method. This avoids unnecessary updates when the page is no longer visible.

Application Lifecycle in Flutter

The **application lifecycle in Flutter** refers to the sequence of states an app goes through during its execution, such as *resumed*, *inactive*, *paused*, and *detached* and the mechanisms provided to handle transitions between these states.

Flutter provides a platform-independent abstraction of the lifecycle through the `AppLifecycleState` enum. Developers can monitor these changes by implementing the `WidgetsBindingObserver`, which allows the application to listen for lifecycle updates.

In particular, the method `didChangeAppLifecycleState(AppLifecycleState state)` is overridden to react to state transitions, enabling behaviors such as pausing tasks, saving data, or managing background operations when the app moves between foreground and background.

Difference between permission denied and GPS disabled

The two error conditions shown in the diagram represent different problems.

The `PERMISSION_DENIED` state means that the application is not allowed to access the device location. In this case, the problem is related to the application permission, so the user must enable the permission from the app settings.

The `GPS_DISABLED` state means that the application has permission to access location, but the location service of the device is currently turned off. In this case, the problem is not related to the app permission, but to the system location service, so the user must enable GPS from the device settings.

For this reason, the page provides two different actions: opening the application settings in the case of denied permission, and opening the location settings in the case of disabled GPS.


View Compass and Altitude Information Sequence Diagram
User
Geowatch
servicecontroller
geoservice
permissionservice
Open navigation page
initState()
addObserver(this)
_initLocation()
loadNavigationLocation()
isLocationGranted()
bool
alt [Permission denied]
NavigationLocationState(error = "PERMISSION_DENIED")
setState(_error, _isChecking, _altitude = null, _position = null)
Show permission error state
opt [User presses open settings button]
Tap open settings
openPermissionSettings()
openAppSettingsPage()
completed
completed
[Permission granted]
isLocationServiceEnabled()
bool
alt [GPS disabled]
NavigationLocationState(error = "GPS_DISABLED")
setState(_error, _isChecking, _altitude = null, _position = null)
Show GPS disabled error state
opt [User presses open settings button]
Tap open settings
openGpsSettings()
openLocationSettingsPage()
completed
completed
[GPS enabled]
NavigationLocationState(error = null)
dispose _positionSub
navigationPositionStream()
getPositionStream()
Stream<Position>
Stream<Position>
loop [for each position update]
Position
setState(_altitude, _position, _error = null, _isChecking = false)
Show altitude and coordinates
compassStream()
compassStream()
Stream<double?>
Stream<double?>
loop [for each heading update]
_direction(heading)
Show compass heading and direction
opt [App returns to foreground]
didChangeAppLifecycleState(resumed)
setState(_isChecking = true)
_initLocation()

2.4.9 Login Sequence Diagram

The login process supports two authentication mechanisms: email/password and Google sign-in. These are modeled as alternative flows within the same sequence diagram.

Email and password login When the user presses the login button, the page executes the method `_loginEmailPwd(context)`. The method first validates that both email and password are not empty. If one of the two fields is missing, the page shows an error message through a `SnackBar`.

If the input is valid, the page invokes `userController.login(email, password)`. The `UserController` delegates the operation to the `AuthService`, which uses Firebase Authentication to authenticate the user through `signInWithEmailAndPassword()`.

Firebase Authentication is used exclusively to verify user identity and retrieve a unique user identifier (UID). The method `signInWithEmailAndPassword()` returns a `UserCredential` object, which contains the authenticated Firebase user. After authentication, the system invokes `loadUserCore()`, which retrieves the user document from Firestore:

- Username
- Email
- Profile data
- Level and statistics

This method intentionally avoids loading relational or large data such as followers, following, or diaries. This design prevents blocking the UI during login and improves perceived performance.

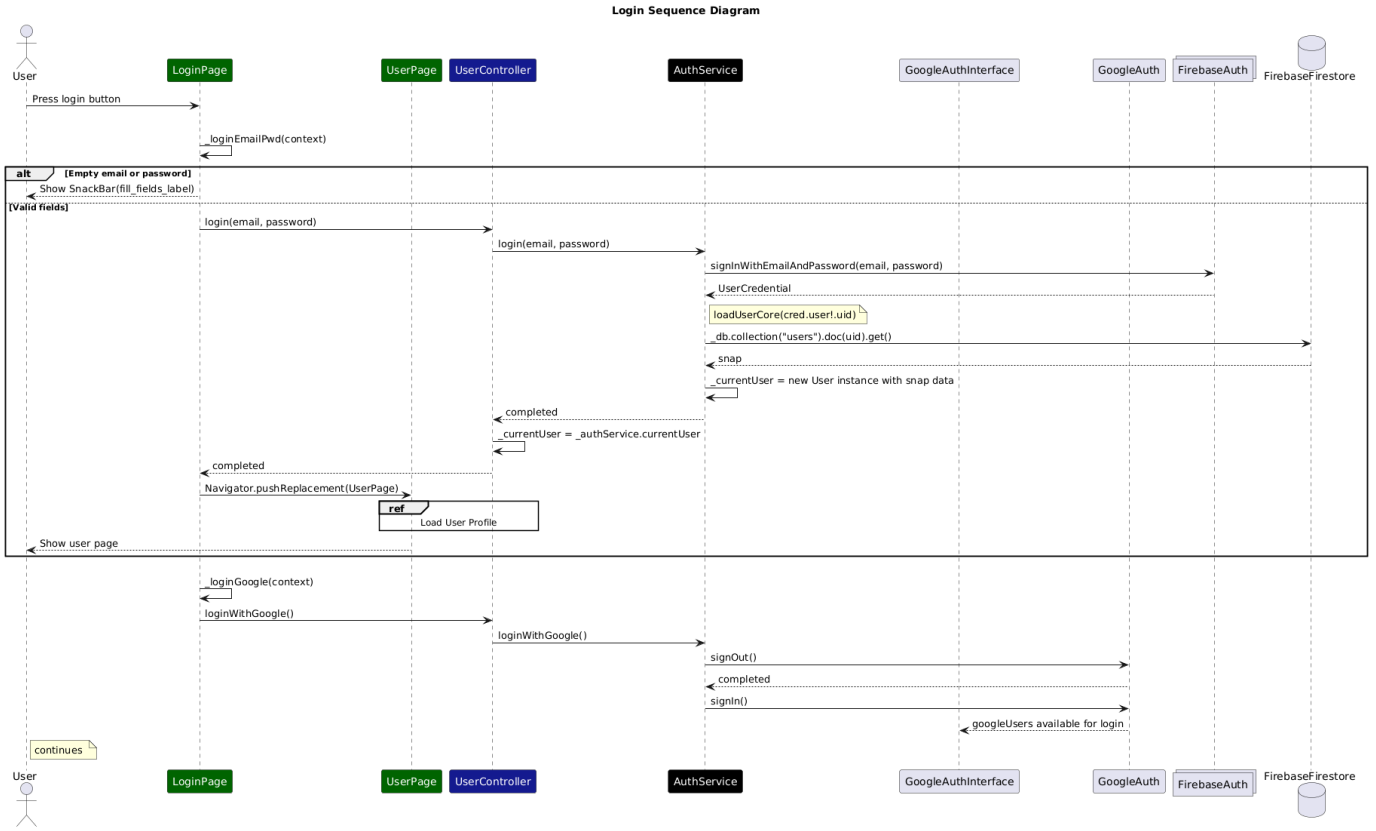
Instead, additional data is loaded lazily in the UserPage using `FutureBuilder`, enabling asynchronous and incremental UI updates.

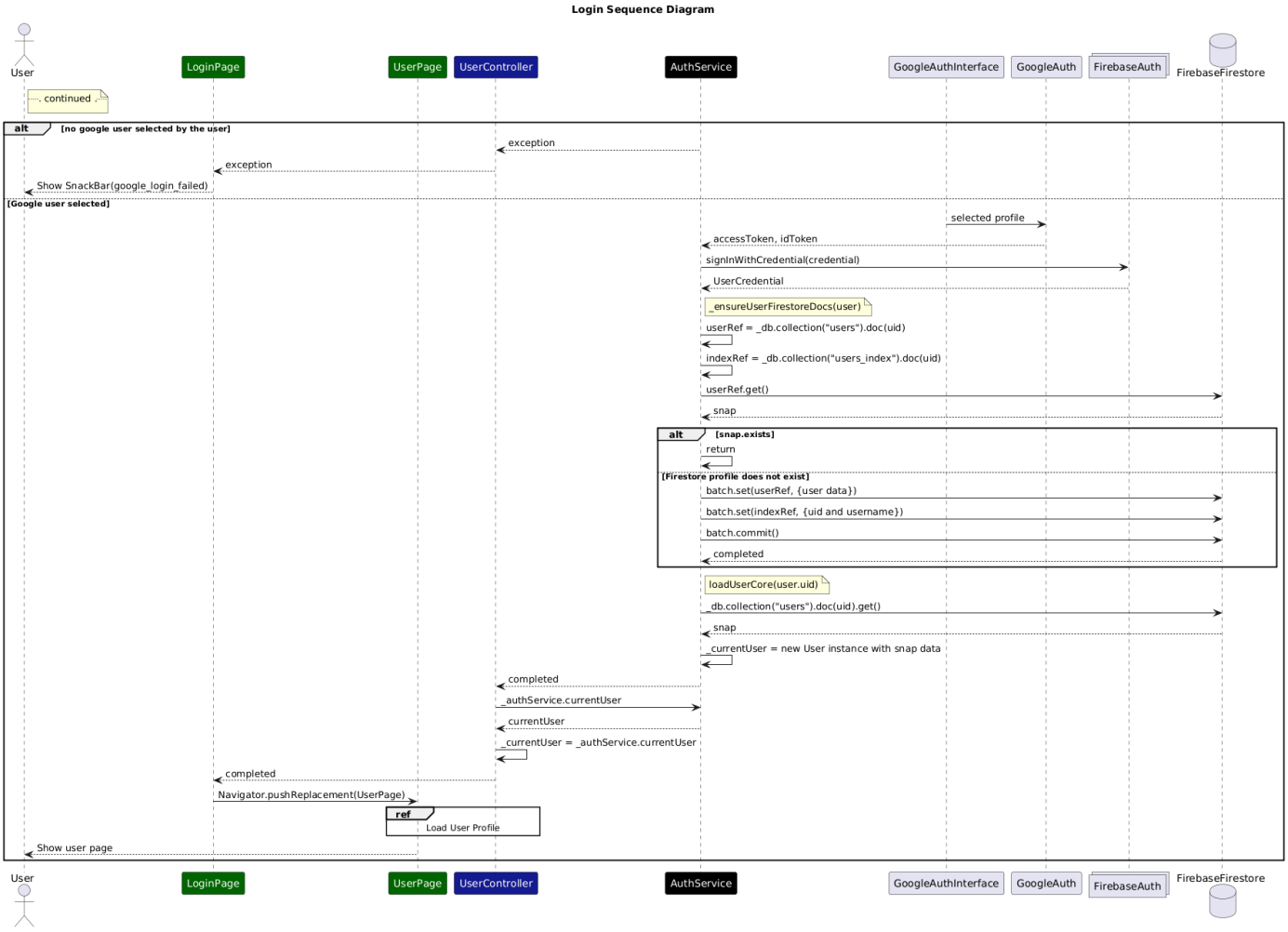
After successful login, the application navigates to the UserPage, whose loading behavior is described in a separate sequence diagram.

Google login When the user presses the Google Sign-In button, the page executes `_loginGoogle(context)`. The request is delegated to `userController.loginWithGoogle()`, which calls the corresponding method in the `AuthService`.

The Google login flow first forces a sign-out from the previous Google session and then opens the Google account interface. If the user does not select a profile, the flow terminates with an error message shown in the UI.

If the user selects a Google account, the `AuthService` retrieves the Google authentication tokens and exchanges them for Firebase credentials through `signInWithCredential()`. Once Firebase authentication succeeds, the service verifies whether the user already has a corresponding Firestore profile through `_ensureUserFirestoreDocs(user)`. If the profile does not exist, the service creates the required documents in both the `users` and `users_index` collections. After that, the method `loadUserCore(uid)` is executed, and the page navigates to the `UserPage`.


Login Sequence Diagram
User
LoginPage
UserPage
UserController
AuthService
GoogleAuthInterface
GoogleAuth
FirebaseAuth
FirebaseFirestore
Press login button
_loginEmailPwd(context)
alt [Empty email or password]
Show SnackBar(fill_fields_label)
[Valid fields]
login(email, password)
login(email, password)
signInWithEmailAndPassword(email, password)
UserCredential
loadUserCore(cred.user!.uid)
_db.collection("users").doc(uid).get()
snap
_currentUser = new User instance with snap data
completed
_currentUser = _authService.currentUser
completed
Navigator.pushReplacement(UserPage)
ref
Load User Profile
Show user page
_loginGoogle(context)
loginWithGoogle()
loginWithGoogle()
signOut()
completed
signIn()
googleUsers available for login
continues


Login Sequence Diagram
User
LoginPage
UserPage
UserController
AuthService
GoogleAuthInterface
GoogleAuth
FirebaseAuth
FirebaseFirestore
continued
alt [no google user selected by the user]
exception
exception
Show SnackBar(google_login_failed)
[Google user selected]
selected profile
accessToken, idToken
signInWithCredential(credential)
UserCredential
_ensureUserFirestoreDocs(user)
userRef = _db.collection("users").doc(uid)
indexRef = _db.collection("users_index").doc(uid)
userRef.get()
snap
alt [snap.exists]
return
[Firestore profile does not exist]
batch.set(userRef, {user data})
batch.set(indexRef, {uid and username})
batch.commit()
completed
loadUserCore(user.uid)
_db.collection("users").doc(uid).get()
snap
_currentUser = new User instance with snap data
completed
_authService.currentUser
currentUser
_currentUser = _authService.currentUser
completed
Navigator.pushReplacement(UserPage)
ref
Load User Profile
Show user page

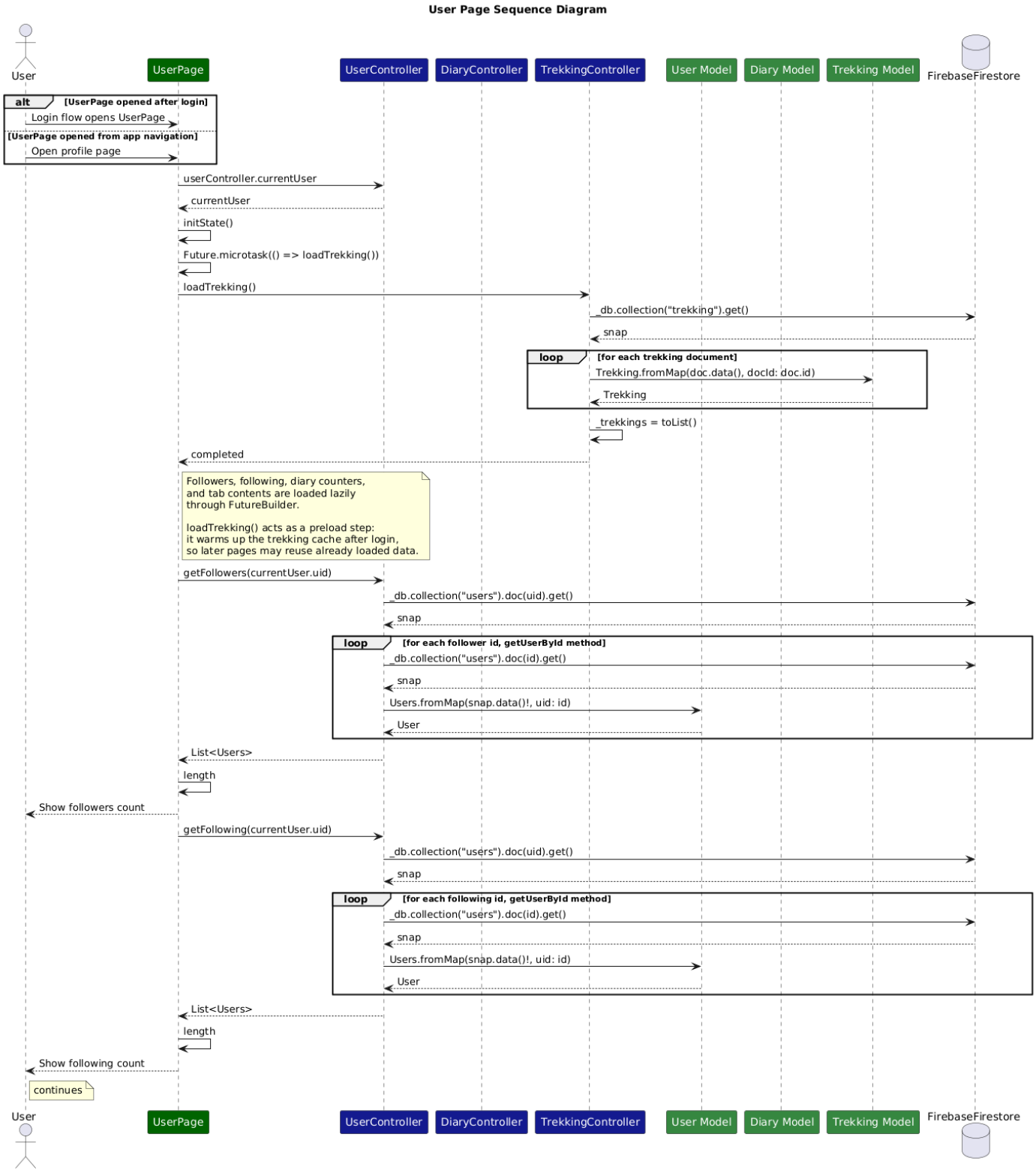
2.4.10 View User Information Sequence Diagram

The UserPage can be opened either as the landing page after a successful login or through the normal application navigation. Once the page is displayed, it retrieves the current authenticated user from the UserController.

During initialization, the page schedules the invocation of `loadTrekking()` through `Future.microtask()`, instead of calling it directly inside `initState()`. This choice postpones the trekking load operation until the initialization phase is completed, keeping the widget initialization lighter. Although trekking loading is not strictly part of the core profile information, this preload can still be justified for the following reasons

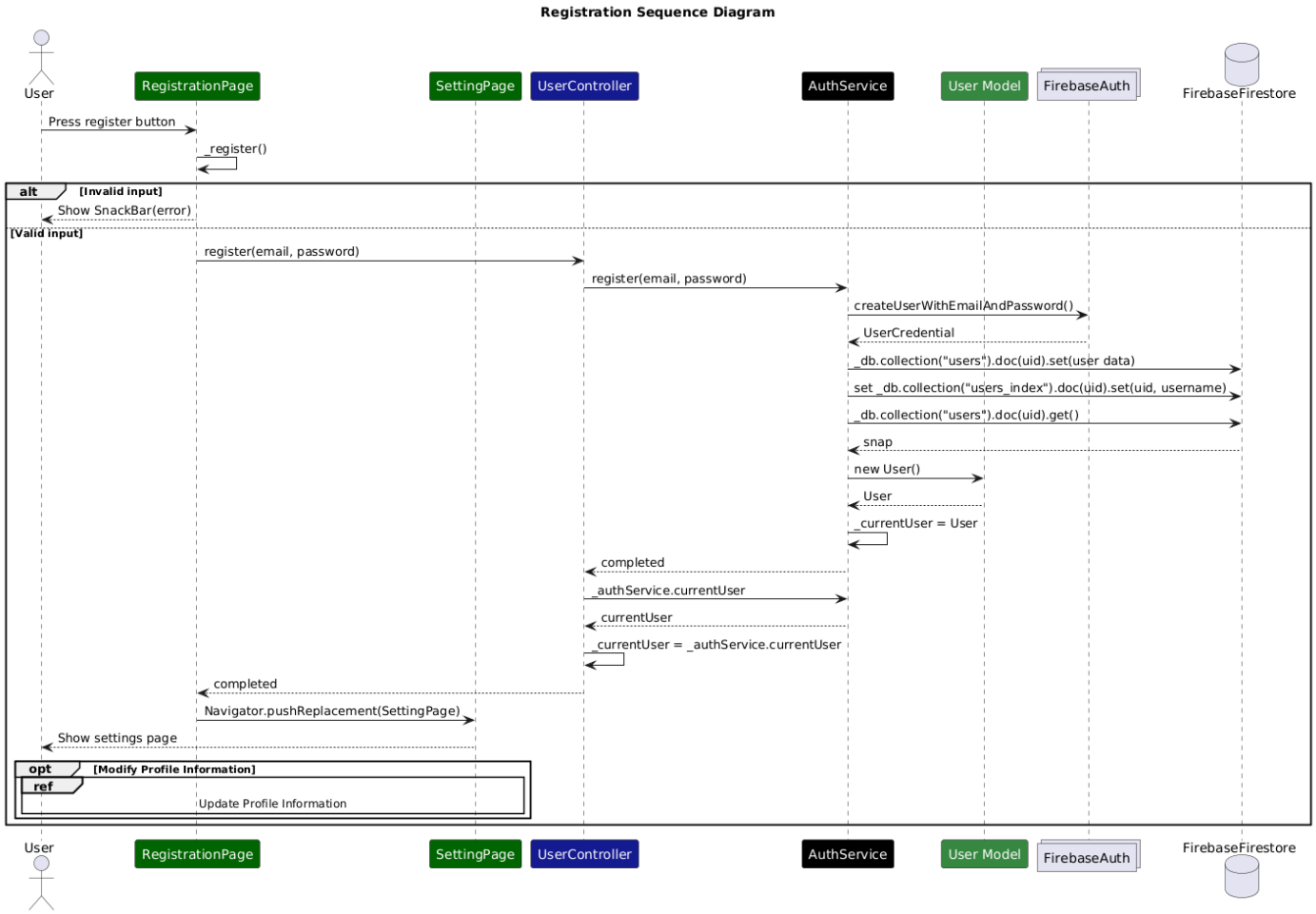
- the UserPage is often the first page shown after login, so loading trekking data here can anticipate work that will likely be needed shortly afterward by pages such as the HomePage, TrekkingPage, SearchPage, and the saved trekking tab itself
- the UserPage allows navigation toward saved trekkings and trekking detail pages, so having the trekking cache already populated may reduce subsequent loading work
- `loadTrekking()` acts as a warm-up step for the controller-side cache, because it populates the local `_trekkings` list and makes data immediately available for methods such as `getTrekkingById()`, `getTrekkingId()`, using Firestore only as secondary solution

The main profile information shown in the page is obtained from the current user already available in the UserController. Additional information is retrieved lazily through `FutureBuilder`. This applies to followers, following users, diary counters, public diaries, private diaries, and saved trekkings. For followers and following, the UserController first retrieves the current user document from Firestore in order to obtain the identifiers stored in the `Followers` and `Following` fields. Then, for each identifier, the controller invokes `getUserById()`, which retrieves the corresponding Firestore document and builds a `Users` object through `Users.fromMap()`. The total diary count is computed through the helper method `_loadAllDiaries(context, uid)` defined in the UserPage. This method invokes both `getPublicDiaries(uid)` and `getPrivateDiaries(uid)` from the DiaryController, then merges the two returned lists into a single collection. For the diary tabs, the DiaryController queries Firestore and converts each result document into a `Diary` object through `Diary.fromMap()`. For the saved trekking tab, the TrekkingController first retrieves the current user document to obtain the identifiers stored in `Saved_trekkings`. Then, for each identifier, it loads the corresponding trekking document and converts it into a `Trekking` object through `Trekking.fromMap()`. This design combines a lightweight preload of trekking data with lazy loading of profile-related collections, keeping the UserPage responsive while still preparing application data that is likely to be useful immediately after login.


User Page Sequence Diagram
User
UserPage
UserController
DiaryController
TrekkingController
User Model
Diary Model
Trekking Model
FirebaseFirestore
alt
[UserPage opened after login]
Login flow opens UserPage
[UserPage opened from app navigation]
Open profile page
userController.currentUser
currentUser
initState()
Future.microtask(() => loadTrekking())
loadTrekking()
_db.collection("trekking").get()
snap
loop
[for each trekking document]
Trekking.fromMap(doc.data(), docId: doc.id)
Trekking
_trekkings = toList()
completed
Followers, following, diary counters,
and tab contents are loaded lazily
through FutureBuilder.
loadTrekking() acts as a preload step:
it warms up the trekking cache after login,
so later pages may reuse already loaded data.
getFollowers(currentUser.uid)
_db.collection("users").doc(uid).get()
snap
loop
[for each follower id, getUserById method]
_db.collection("users").doc(id).get()
snap
Users.fromMap(snap.data()!, uid: id)
User
List<Users>
length
Show followers count
getFollowing(currentUser.uid)
_db.collection("users").doc(uid).get()
snap
loop
[for each following id, getUserById method]
_db.collection("users").doc(id).get()
snap
Users.fromMap(snap.data()!, uid: id)
User
List<Users>
length
Show following count
continues

2.4.11 Registration Sequence Diagram

The registration functionality is handled by the `RegistrationPage`, which delegates the creation of the new account to the `UserController` and the `AuthService`. When the user presses the register button, the page executes the method `_register(context)`. The method first validates the input fields. If one or more fields are empty, the UI shows a `SnackBar` requesting the user to fill all fields. If the password and confirmation password do not match, another `SnackBar` is shown. If the data is valid, the page activates the loading state and calls `userController.register(email, password)`. The `UserController` delegates the operation to the `AuthService`, which creates the account in Firebase Authentication through `createUserWithEmailAndPassword()`. After the account is successfully created, the `AuthService` derives a default username from the email prefix and creates the corresponding Firestore documents in the `users` and `users_index` collections. The first document stores the full user profile data, while the second supports search functionality through a normalized username representation. Once the Firestore documents have been created, the `AuthService` loads the newly created user profile through `loadUserCore(uid)`. The resulting user object is returned to the `UserController`, which updates its internal state. Finally, the `RegistrationPage` shows a success message and navigates to the `SettingPage`. The loading indicator is then disabled. It is important to note that, in the current implementation, the registration page only supports the traditional email/password flow. Google-based account creation is not handled by the registration page itself, but is implicitly managed during the first successful Google login through the creation of the Firestore profile documents.



2.4.12 Update User Information Sequence Diagram

The update of user information is handled by the `SettingPage`, which allows the user to modify several profile attributes such as username, name, surname, birthdate, profile image, and email. The page delegates saving user information to the `UserController`, which is responsible for uploading the local user state to Firestore.

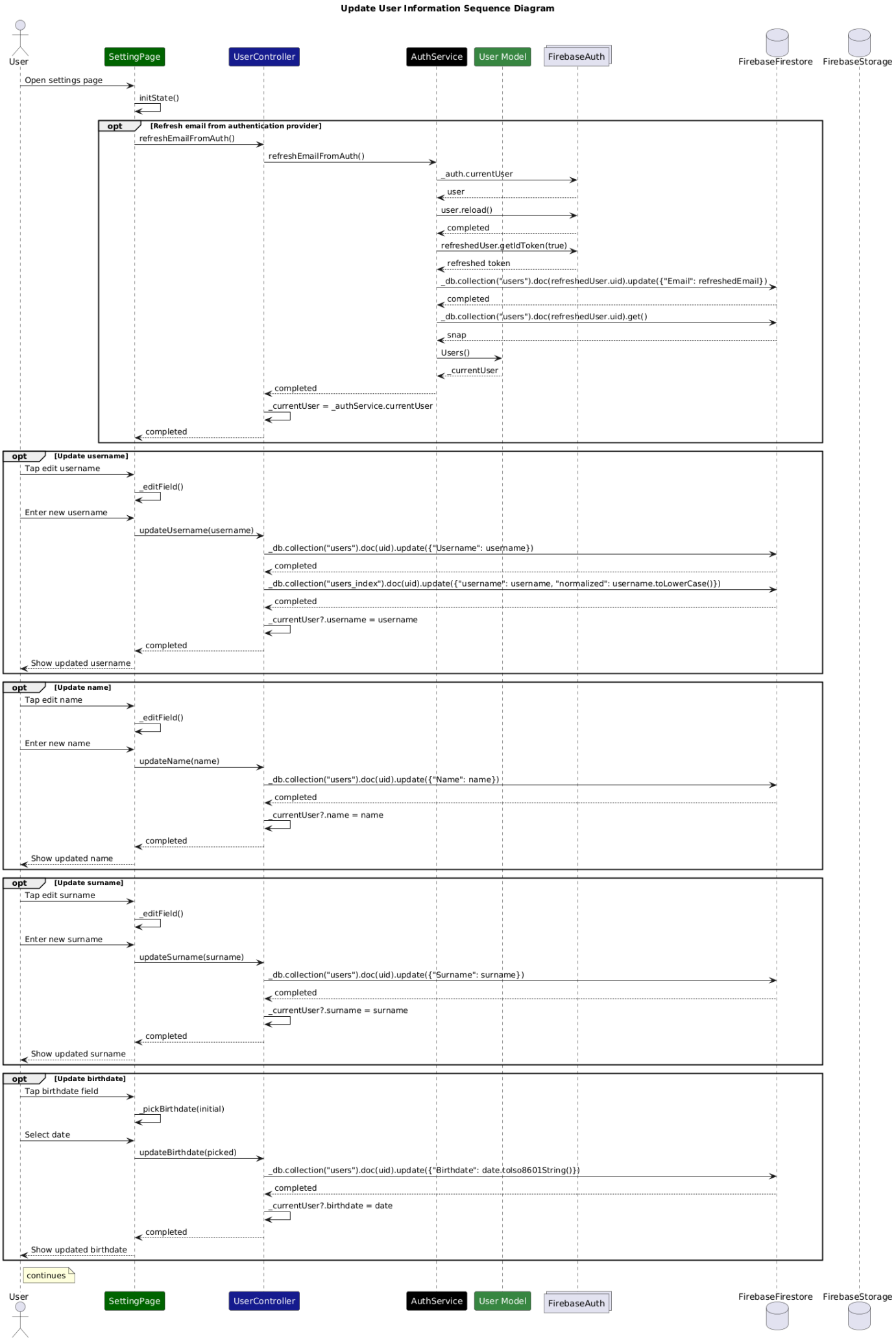
When the page is opened, it executes an initialization step by invoking `refreshEmailFromAuth()`, in order to check that the email stored in Firestore is aligned with the current value available in Firebase Authentication. Infact, if the user has requested to change its email and has verified the new one by clicking the link in the mail inbox, Firebase Auth may have registered that the user is associated to a new email, so this reference shall be updated also in the Firestore user record. The `AuthService` first reloads the authenticated Firebase user, loads the updated email, updates the Firestore user document, and finally rebuilds the local `User` through the method `loadUserCore()`.

The update of textual profile fields, such as username, name, and surname, follows a common pattern. The UI opens a dialog through `_editField()`, the user enters a new value, and the page delegates the save to Firestore operation to the `UserController`. The controller then updates the Firestore document and synchronizes the corresponding field inside the local `_currentUser` object. The username update also requires an update of the `users_index` collection, so that the user remains searchable through the normalized username.

The birthdate update is handled similarly, but the input is collected through `showDatePicker()` instead of a text dialog. Once the user selects a date, the `UserController` updates Firestore and updates the local model field.

The profile image update uses the image picker to let the user select a file from the gallery. The `UserController` uploads the file to Firebase Storage, obtains the public download URL, stores that URL inside the Firestore user document, and updates the local `_currentUser.photoProfile` field. In this way, the UI can show the new profile image after the operation completes.

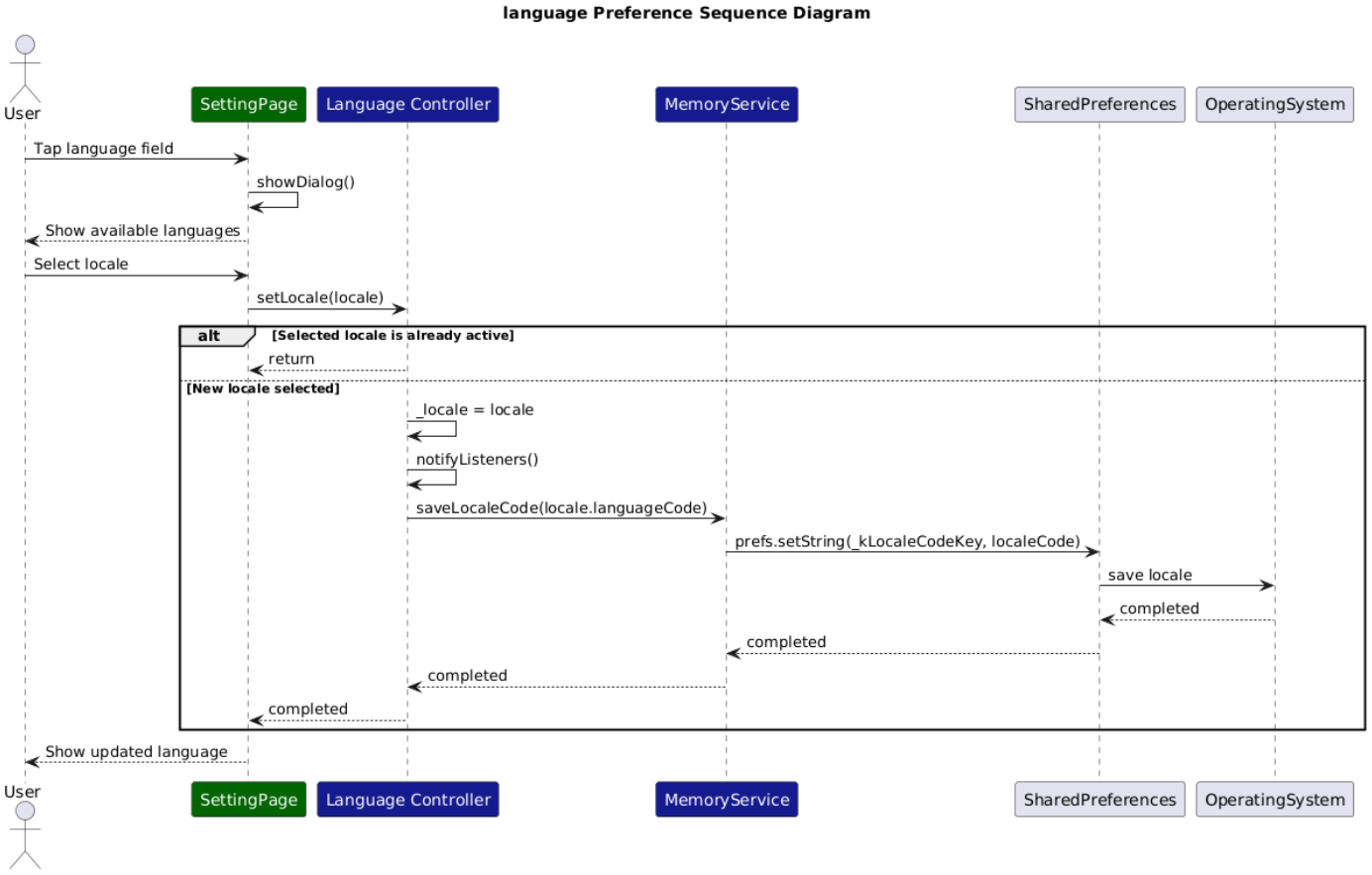
The email update is more complex because it involves Firebase Authentication. This operation is available only for password-based accounts, since Google profiles are managed by the external auth provider GoogleAuth. The page opens a dedicated dialog where the user enters the new email and the current password. The `UserController` delegates the operation to the `AuthService`, which first reauthenticates the current user through `reauthenticateWithCredential()` because, according to Firebase documentation, some security sensitive actions such as setting a new email address require that the user has recently signed in, then invokes `verifyBeforeUpdateEmail(newEmail)`. Firebase Authentication sends a verification link to the new email address. If the request succeeds, the page records that an email change has been requested and informs the user through a confirmation message. If Firebase returns an error, the exception is shown through a `SnackBar`.


Update User Information Sequence Diagram
User
SettingPage
UserController
AuthService
User Model
FirebaseAuth
FirebaseFirestore
FirebaseStorage
Open settings page
initState()
opt [Refresh email from authentication provider]
refreshEmailFromAuth()
refreshEmailFromAuth()
_auth.currentUser
user
user.reload()
completed
refreshedUser.getIdToken(true)
refreshed token
_db.collection("users").doc(refreshedUser.uid).update({"Email": refreshedEmail})
completed
_db.collection("users").doc(refreshedUser.uid).get()
snap
Users()
_currentUser
completed
_currentUser = _authService.currentUser
completed
opt [Update username]
Tap edit username
_editField()
Enter new username
updateUsername(username)
_db.collection("users").doc(uid).update({"Username": username})
completed
_db.collection("users_index").doc(uid).update({"username": username, "normalized": username.toLowerCase()})
completed
_currentUser?.username = username
completed
Show updated username
opt [Update name]
Tap edit name
_editField()
Enter new name
updateName(name)
_db.collection("users").doc(uid).update({"Name": name})
completed
_currentUser?.name = name
completed
Show updated name
opt [Update surname]
Tap edit surname
_editField()
Enter new surname
updateSurname(surname)
_db.collection("users").doc(uid).update({"Surname": surname})
completed
_currentUser?.surname = surname
completed
Show updated surname
opt [Update birthdate]
Tap birthdate field
_pickBirthdate(initial)
Select date
updateBirthdate(picked)
_db.collection("users").doc(uid).update({"Birthdate": date.toIso8601String()})
completed
_currentUser?.birthdate = date
completed
Show updated birthdate
continues


Update User Information Sequence Diagram
User
SettingPage
UserController
AuthService
User Model
FirebaseAuth
FirebaseFirestore
FirebaseStorage
continued
opt [Update profile photo]
Tap edit profile image
_pickProfileImage()
Open image picker
Select image
updateProfilePhoto(File)
ref().child("profile_photos").child(uid).child("$uid.jpg").putFile(image)
completed
ref.getDownloadURL()
url
_db.collection("users").doc(uid).update({"Photo_profile": url})
completed
_currentUser?.photoProfile = url
completed
Show updated profile image
opt [Restore Google profile photo if a Google user has modified the picture]
Tap restore Google photo
restoreGoogleProfilePhoto()
currentUid
uid
currentPhotoUrl
photoUrl
_db.collection("users").doc(uid).update({"Photo_profile": _authService.currentPhotoUrl})
completed
Firebase Auth stores the photoURL of the Google profile in the user profile after login.
_currentUser?.photoProfile = _authService.currentPhotoUrl
completed
Show restored Google photo
opt [Update email for password user]
Tap edit email
_showChangeEmailDialog()
Enter new email and current password
changeEmail(newEmail, currentPassword)
changeEmail(newEmail, currentPassword)
_auth.currentUser
user
EmailAuthProvider.credential(email, currentPassword)
credential
user.reauthenticateWithCredential(credential)
completed
user.verifyBeforeUpdateEmail(newEmail)
verification email sent
alt
completed
completed
setState(_emailChangeRequested = true)
Show confirmation SnackBar
[Email update error]
ChangeEmailException(code)
ChangeEmailException(code)
Show error SnackBar
opt [Password reset request]
Tap send password reset
requestPasswordReset()
requestPasswordReset()
_auth.sendPasswordResetEmail(email: _currentUser!.email)
completed
completed
completed
Show SnackBar(password reset sent)
opt [Change language]
Tap language field
ref
Language preference

Change Language Sequence Diagram

The language change process is handled by the `Language` controller, and saving the preference on the phone memory is delegated to the `MemoryService`. When the user selects the language field in the SettingPage, the application opens a dialog that displays the list of available locales. Once the user chooses a language, the dialog invokes `setLocale(locale)` on the Language controller. The controller first checks if the selected locale is already active. If so, no further action is needed. Otherwise, it updates the internal `_locale` field, calls `notifyListeners()`, and then delegates the saving the preference to the `MemoryService` through `saveLocaleCode(locale.languageCode)`. The MemoryService stores the selected language code locally using shared preferences. This allows the application to restore the previously selected language when it is reopened, using the method `loadSavedLocale()`.



2.4.13 Phone Watch Pair

High level description of the pairing mechanism

The pairing mechanism associates a Wear OS device with an authenticated mobile user through a backend-mediated approval process. The watch does not authenticate directly with Firebase Auth; instead, device identity and user identity are linked via Firestore.

Device Identity

Each watch either makes or restores from memory a `watchId`, stored securely on-device. A corresponding document exists in Firestore:

`watch/{watchId}`

The `watchId` uniquely identifies the physical device and is never deleted during logout.

Pairing Flow

1. Initialization (Watch) The watch provides a new random UUID (`qrToken`) and writes:

`watch_pair/{watchId}`

with:

- `status = "waiting"`
- `uid = null`
- `qrToken`
- `createdAt`, `expiresAt`

A QR code encoding `watchId` and `qrToken` is displayed. URI format:
`triplo://watch-pair/<watchId>?t=<token>`

2. Approval (Phone) After scanning the QR, the authenticated mobile app:

- Verifies token validity and expiration.
- Updates the document with:
  - `status = "approved"`
  - `uid = request.auth.uid`

3. Completion (Watch) The watch listens to the document in real time. When `status == "approved"` and `uid` is set, pairing is considered complete and the watch loads user data using that `uid`.

Session Restoration

On startup, the watch checks:

`watch_pair/{watchId}`

If the document is already approved, the session is restored.

Logout

Logout does not delete the device identity. Instead, the watch resets:

- `status = "waiting"`
- `uid = null`

This preserves device identity while removing user association.

Security Properties

- Only authenticated users may approve pairing.
- QR tokens are random and time-limited.

This design cleanly separates device identity from user authentication while minimizing credential exposure on the wearable device.

Phone-side smartwatch pairing approval from the SettingPage component

Figure ?? illustrates the phone-side portion of the smartwatch pairing. In the phone UI implementation, the QR scanning interaction belongs to the `SettingPage` component.
The interaction starts when the user opens the pairing section and scans the QR code on the smartwatch. Once a QR code is detected, the page extracts the raw payload and checks if that code has already been processed. This behaviour is handled by the boolean flag `_handled`.
This flag is used to not allow repeated processing of the same scanner detection while the page is still active.
If a valid raw payload is available, `SettingPage` delegates the parsing step to `UserController.extractWatchPa` which delegates the actual logic of the phone side pairing process to `AuthService.extractWatchPair()`.
This service parses the scanned URI and verifies that it matches the following structure:
`triplo://watch-pair/<watchId>?t=<token>`
The service also validates the presence of both the smartwatch identifier and the temporary QR token in the URI payload. If the payload is malformed, the page shows an error message and enables a new scan by resetting `_handled`.
If the QR payload is valid, the page shows a confirmation dialog to the user. If the user does not confirm the operation, the page allows the next scan. If the user confirms, the page invokes `UserController.approveWatchPair()`, which forwards the request to `AuthService.approveWatchPair()`.
The actual approval is performed inside `AuthService`, since this step depends on the currently authenticated phone user and on the pairing state stored in Firestore. The service executes a Firestore transaction on the document `watch_pair/<watchId>`. The transaction reads the document snapshot, denoted as `snap`, and validates the pairing request by checking four conditions: the document must exist, its status must be `waiting`, the token stored in Firestore must match the token contained in the scanned QR code, and the expiration timestamp must still be valid.
Only if all these conditions hold, the transaction updates the pairing document by setting `status = approved` and by storing the UID of the authenticated phone user. From an architectural perspective, the transaction is important because it makes validation and approval atomic. This reduces the risk of inconsistent state transitions. The page itself does not directly interact with Firestore or Firebase Authentication; it only reacts to success or failure by updating local UI state.

Watch-side access to the user area through the pairing flow

Figure ?? describes how the smartwatch application grants access to the user area through the pairing mechanism. In the watch app implementation, the user does not open a pairing screen, the interaction starts from `NavigationPage`: when the user taps the `User` button, the application performs a `Navigator.push()` toward `PairingGateway(child: UserPage())`.

Pairing Gateway

From an architectural perspective, `PairingGateway` should not be interpreted as an autonomous functional component, it is considered part of the PairingService and its role is to decide whether the watch should render the QR-based pairing screen or the actual `UserPage`.
The pairing bootstrap starts in the state of `PairingGateway`, which is the widget pushed when the user opens the user area from `NavigationPage`. Its responsibility is not to implement the pairing itself, but to bootstrap the pairing state and decide which page should be rendered, either the `LoginPage` or `UserPage`.

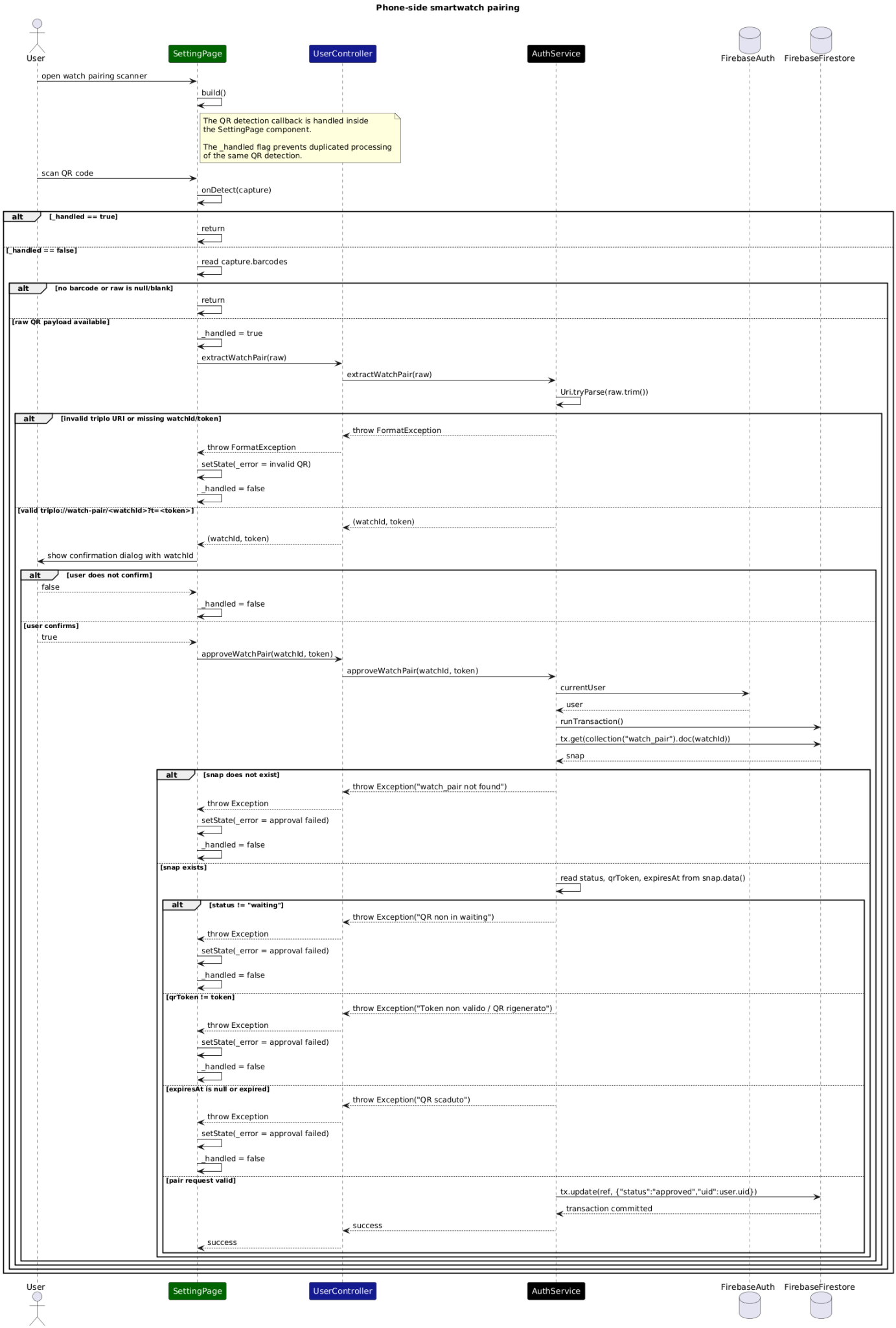
Restoring the session if the user has already paired the watch

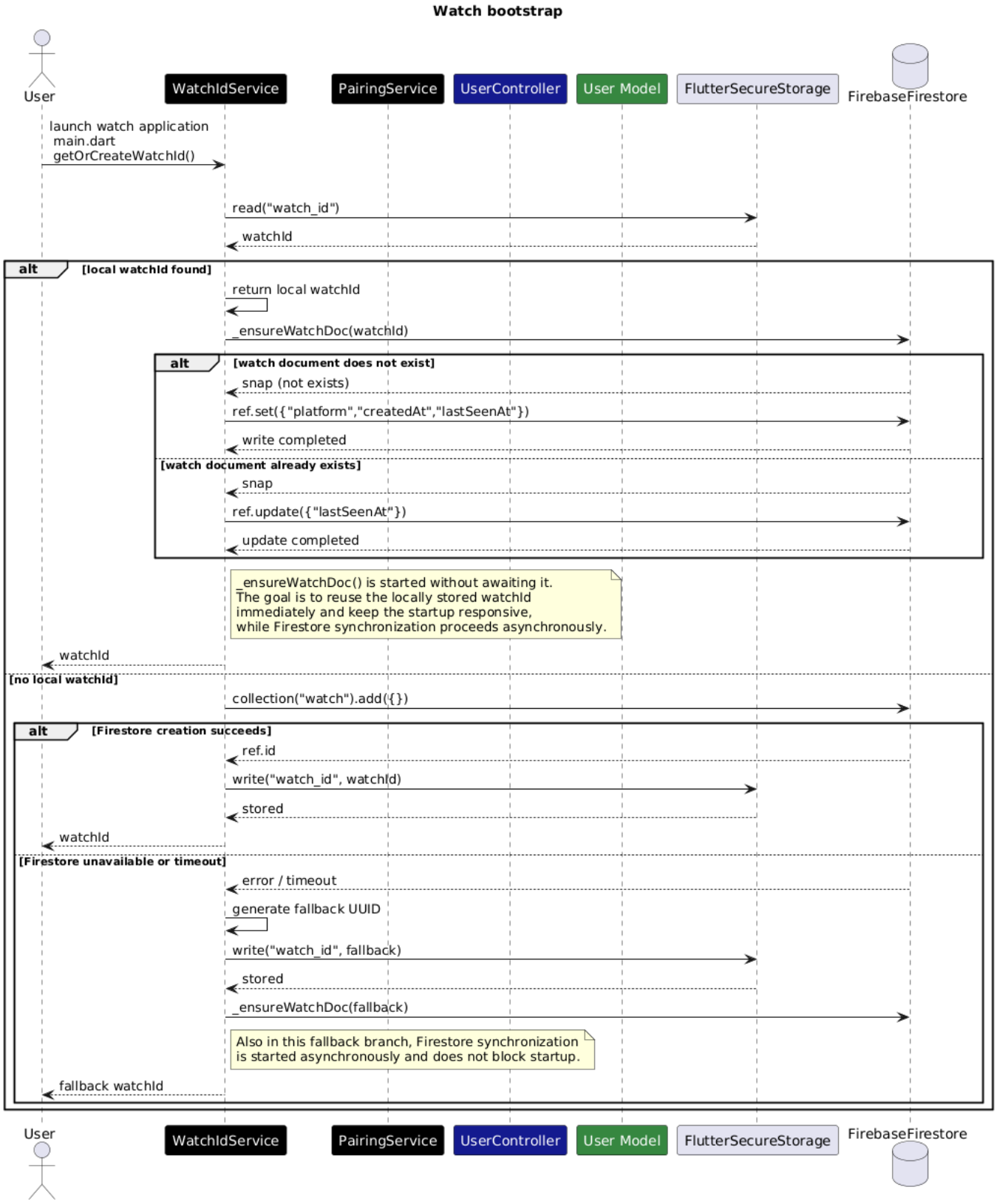
The pairing logic relies on two complementary mechanisms. First, `PairingService.restoreWatchPairing()` performs a read of the Firestore document `watch_pair/<watchId>` in order to determine the current pairing state. Second, the service installs a real-time subscription through `docRef.snapshots().listen()`, so that later updates to the same Firestore document, such as approval from the phone or a reset to the `waiting` state, are seen by the watch without requiring to implement a more complicated ping-like mechanism to listen to updates of the document on Firestore.
If the pairing document is already in `approved` state and contains a non-empty UID, `PairingService` stores that UID in its internal variable `_pairedUid` and calls `notifyListeners()`. In this context, the listeners are the Flutter widgets that observe `PairingService` through Provider, notably `PairingGateway`.
At this point, `PairingGateway` requests `UserController.loadCurrentPairedUser()`. This step is necessary because the pairing service only knows which user UID is associated with the watch, but it does not load the complete user profile. The actual profile loading is delegated to `UserController`, which reads the corresponding document from the `User` collection by calling `loadUserCore(uid)`. The returned Firestore snapshot, denoted as `snap`, is converted into a `User` domain object through `Users.fromMap()` and stored in `UserController` as `_currentUser`. When `loadUserCore()` completes, `UserController` also calls `notifyListeners()`.
Here again, the listeners are the widgets that observe `UserController` through Provider and the `PairingGateway` is a listener.
Then the UI is rebuilt because the `PairingGateway` returns the widget child that was set by the `Navigation Page` through `Navigator.push()` toward `PairingGateway(child: UserPage())`. The `UserPage` is then able to see that a paired UID is available.
In this sense, the pairing service is responsible for determining whether the watch is associated with a user, whereas the user controller is responsible for loading the actual user data needed by `UserPage`.

Pair the watch if the user has not paired it

If no approved pairing is found, the `PairingService` starts a new pairing cycle through `PairingService.start`
In this branch, the service makes a new QR token, stores it in local state, writes the Firestore document `watch_pair/<watchId>` with status `waiting`, a null UID, an expiration timestamp, and a new token, and then installs a snapshot listener on that document. A local expiration timer is also started. The `PairingService` then renders `LoginPage`, which reads the QR payload exposed by `PairingService` and displays it to the user. The `LoginPage` is only the visual pairing screen, it does not control the pairing itself.
The most relevant transition regards the phone approving the QR code. This operation updates the Firestore document `watch_pair/<watchId>` by setting its status to `approved` and assigning the cor-

responding user UID. The update is received by the active snapshot listener previously installed by `PairingService.startWatchPairing()` through `docRef.snapshots().listen()`.
Inside the listener, the service reads the updated document and retrieves the fields `status`, `uid`, and `qrToken`. At this point, the service compares the token stored in Firestore with the locally active token (`_qrToken`). This comparison is performed inside the listener itself through the instruction `if (tokenOnDb != _qrToken) return;` and ensures that only updates referring to the currently displayed QR code are accepted. If the tokens do not match, the update is ignored, in order not to approve stale QR codes.
If the token matches and the status is `approved`, `PairingService` stores the paired UID in `_pairedUid`, stops the expiration timer, and calls `notifyListeners()`. This notification is sent over to the `PairingGateway`, that rebuilds the UI. The `PairingGateway` reacts to the new state by invoking `UserController.loadCurrentPa` which loads the corresponding user document from Firestore and reconstructs the `Users` model. As a result, the UI transitions from the pairing screen to `UserPage`.
The logout and re-pairing flow clarifies an additional architectural point. When the user logs out from the smartwatch user page, `PairingService.logoutWatch()` clears the active timer, stops the current Firestore pairing subscription, resets the pairing document by making a new QR token and restoring the `waiting` state, and clears the locally paired UID. However, the reset of the backend document alone is not sufficient, the observation of the pairing document must be reinstalled through a new call to `restoreWatchPairing()`. This step is crucial because otherwise the watch could display a new QR code while failing to display the UserPage correctly after the first approval performed by the phone.


Phone-side smartwatch pairing
User
SettingPage
UserController
AuthService
FirebaseAuth
FirebaseFirestore
open watch pairing scanner
build()
The QR detection callback is handled inside the SettingPage component.
The _handled flag prevents duplicated processing of the same QR detection.
scan QR code
onDetect(capture)
alt [_handled == true]
return
[_handled == false]
read capture.barcodes
alt [no barcode or raw is null/blank]
return
[raw QR payload available]
_handled = true
extractWatchPair(raw)
extractWatchPair(raw)
Uri.tryParse(raw.trim())
alt [invalid triplo URI or missing watchId/token]
throw FormatException
throw FormatException
setState(_error = invalid QR)
_handled = false
[valid triplo://watch-pair/<watchId>?t=<token>]
(watchId, token)
(watchId, token)
show confirmation dialog with watchId
alt [user does not confirm]
false
_handled = false
[user confirms]
true
approveWatchPair(watchId, token)
approveWatchPair(watchId, token)
currentUser
user
runTransaction()
tx.get(collection("watch_pair").doc(watchId))
snap
alt [snap does not exist]
throw Exception("watch_pair not found")
throw Exception
setState(_error = approval failed)
_handled = false
[snap exists]
read status, qrToken, expiresAt from snap.data()
alt [status != "waiting"]
throw Exception("QR non in waiting")
throw Exception
setState(_error = approval failed)
_handled = false
[qrToken != token]
throw Exception("Token non valido / QR rigenerato")
throw Exception
setState(_error = approval failed)
_handled = false
[expiresAt is null or expired]
throw Exception("QR scaduto")
throw Exception
setState(_error = approval failed)
_handled = false
[pair request valid]
tx.update(ref, {"status":"approved","uid":user.uid})
transaction committed
success
success


Watch bootstrap
User
WatchIdService
PairingService
UserController
User Model
FlutterSecureStorage
FirebaseFirestore
launch watch application
main.dart
getOrCreateWatchId()
read("watch_id")
watchId
alt
[local watchId found]
return local watchId
_ensureWatchDoc(watchId)
alt
[watch document does not exist]
snap (not exists)
ref.set({"platform","createdAt","lastSeenAt"})
write completed
[watch document already exists]
snap
ref.update({"lastSeenAt"})
update completed
_ensureWatchDoc() is started without awaiting it.
The goal is to reuse the locally stored watchId
immediately and keep the startup responsive,
while Firestore synchronization proceeds asynchronously.
watchId
[no local watchId]
collection("watch").add({})
alt
[Firestore creation succeeds]
ref.id
write("watch_id", watchId)
stored
watchId
[Firestore unavailable or timeout]
error / timeout
generate fallback UUID
write("watch_id", fallback)
stored
_ensureWatchDoc(fallback)
Also in this fallback branch, Firestore synchronization
is started asynchronously and does not block startup.
fallback watchId


Watch-side pairing
User
NavigationPage
Pairing Gateway
LoginPage
UserPage
WatchIdService
PairingService
UserController
User Model
FlutterSecureStorage
FirebaseFirestore
PairingGateway(child: UserPage()).
PairingGateway is not as a separate architectural component from PairingService, but its bootstrap and routing logic are represented here because they determine whether the watch shows LoginPage or UserPage.
tap User button
Navigator.push(PairingGateway(UserPage()))
restoreWatchPairing()
collection("watch_pair").doc(watchId).get()
snap
alt
[pairing already approved]
read status and uid from snap.data()
_pairedUid = uid
docRef.snapshots().listen()
subscription installed
true
loadCurrentPairedUser()
collection("users").doc(uid).get()
snap
Users.fromMap(snap.data()!, uid: uid)
Users
_currentUser = Users
success
build child page
show paired user profile
[pairing not approved]
docRef.snapshots().listen()
subscription installed
false
startWatchPairing(forceNew = true)
generate new QR token
_pairedUid = null
_qrToken = token
_pairId = token
_pairCreatedAtLocal = now
collection("watch_pair").doc(watchId).set({status="waiting", qrToken=token, uid=null})
write completed
start expiry timer
docRef.snapshots().listen()
subscription installed
QR state ready
build pairing screen
pairing.qrPayload
qrPayload
show QR code
Phone approves the QR code and pairs the watch
snapshot update(status="approved", uid, qrToken)
compare tokenOnDb with _qrToken
alt
[token matches active QR and status == "approved"]
stop expiry timer
_pairedUid = uid
notifyListeners()
loadCurrentPairedUser()
collection("users").doc(uid).get()
snap
Users.fromMap(snap.data()!, uid: uid)
Users
_currentUser = Users
success
rebuild child page
show paired user profile
[token mismatch or invalid state]
ignore update / keep current UI
opt
[logout on the smartwatch]
logout
logoutWatch()
stop expiry timer
stop pairing subscription
resetPairingOnLogout(regenerateQr = true)
generate new QR token
collection("watch_pair").doc(watchId).set({status="waiting", qrToken=newToken, uid=null})
write completed
_pairedUid = null
_pairingError = null
notifyListeners()
restoreWatchPairing()
collection("watch_pair").doc(watchId).get()
snap
docRef.snapshots().listen()
subscription installed
waiting state restored
build pairing screen
show new QR

2.5 Component interfaces

2.5.1 Component Interfaces Smartphone Application

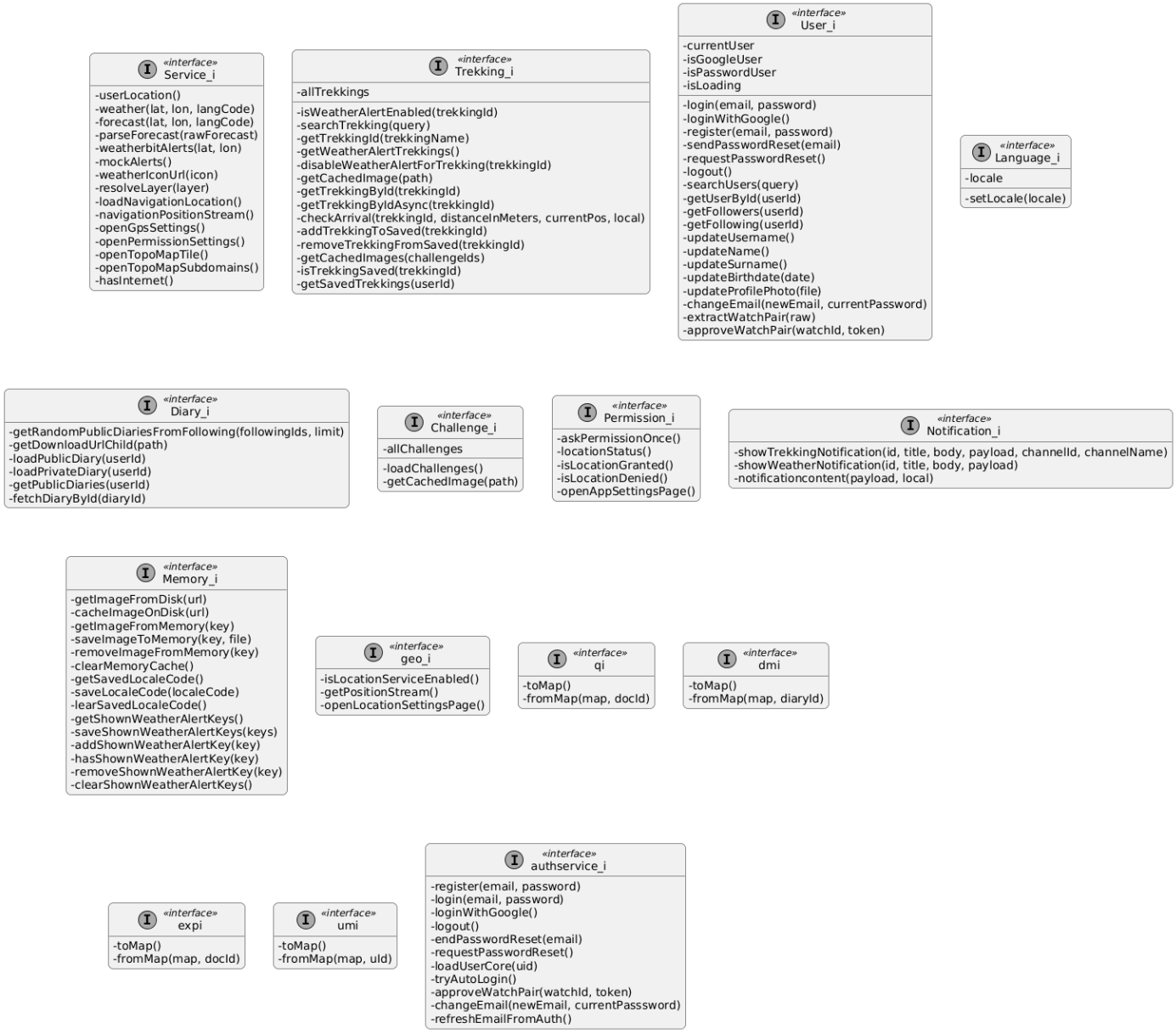


Figure 2.6: Interface diagram smartphone application

The interfaces shown in Figure 2.5.1 should not be interpreted as direct one-to-one representations of language-level constructs. Rather, they are used as an architectural abstraction to explicitly model the set of operations that each component exposes to the rest of the system. Figure 2.5.1 presents the main interfaces exposed by the application components. These interfaces define the contracts through which the different parts of the system interact, making the architecture more modular.

The diagram includes a first group of interfaces associated with the core application domains.

`User_i` defines the operations related to user profile management, following and followers data.
`Trekking_i` groups the operations required to retrieve trekking data, search routes, manage saved treks.
`Diary_i` exposes the operations related to diary retrieval and visualization.
`Challenge_i` provides access to challenge data.
Finally, `Language_i` defines the minimal contract required to manage the application locale.

A second group of interfaces regards infrastructure and platform interaction. `Service_i` acts as

the unified access point for a heterogeneous set of services that are not strictly tied to a single domain entity. As shown in the diagram, this interface includes operations for weather retrieval, forecast parsing, weather alerts, navigation-related location handling, access to map tiles, and redirection to system configuration components such as GPS and permission settings. For this reason, `Service_i` can be interpreted as a façade-oriented interface, collecting in a single access point the functionalities needed by the UI when interacting with external services and operating-system-level capabilities.

The operating system services are further represented through dedicated interfaces, namely `Permission_i`, `Notification_i`, `Memory_i`, and `geo_i`. These interfaces isolate the logic that depends on Flutter packages that communicate with device APIs or with the operating system. This design choice is particularly relevant from a maintenance point of view: if a package or platform-specific integration must be replaced, the impact can be limited to the corresponding service implementation, without requiring modifications in the rest of the application, as explained in section 2.7.4. In this sense, the interfaces shown in Figure 2.5.1 contribute to preserving a clear separation between application logic and platform-dependent concerns.

More specifically, `Permission_i` encapsulates permission requests.
`geo_i` abstracts location-service availability and position streaming
`Notification_i` defines the operations needed to issue local notifications
and `Memory_i` centralizes local storage concerns, including image caching, saving the preferred locale.

The diagram also contains a set of lightweight model-oriented interfaces, namely `qi`, `dmi`, `expi`, and `umi`. These define conversion contracts based on `toMap()` and `fromMap(...)` operations. Their purpose is to standardize the serialization and deserialization logic used to map application objects to and from Firestore representations.

In addition to the previously described interfaces, Figure 2.5.1 also includes the `authservice_i` interface, which encapsulates the operations related to user authentication and credential management.

This interface defines a dedicated contract for handling authentication workflows, including registration, login (both with email/password and Google Sign-In), logout, password reset, and automatic session restoration. Furthermore, it includes operations for account-related updates, such as email changes and smartwatch pairing approval.

From an architectural perspective, `authservice_i` plays a key role in separating authentication from the higher-level domain logic exposed by `User_i`. While `User_i` provides user-related functionalities at the application level, `authservice_i` is responsible for interacting with the underlying authentication provider (e.g., Firebase Authentication), thus centralizing all authentication-related operations in a single component.

This design choice improves modularity and maintainability, as it prevents other components from directly depending on authentication mechanisms and allows the authentication logic to be modified or extended with minimal impact on the rest of the system.

Overall, the interfaces represented in Figure 2.5.1 document a design in which each major responsibility is accessed through a clearly delimited contract. This improves readability of the architecture, supports separation of concerns, and makes explicit which services are expected from each component without exposing internal implementation details. For these reasons, the interface view complements the component decomposition by showing not only which elements exist in the system, but also how their responsibilities are formally made available to the rest of the application.

2.5.2 Component interfaces smartwatch application

2.6 Selected architectural styles and patterns

2.6.1 Architectural Patterns: Model–View–Controller with Mediator and Observer

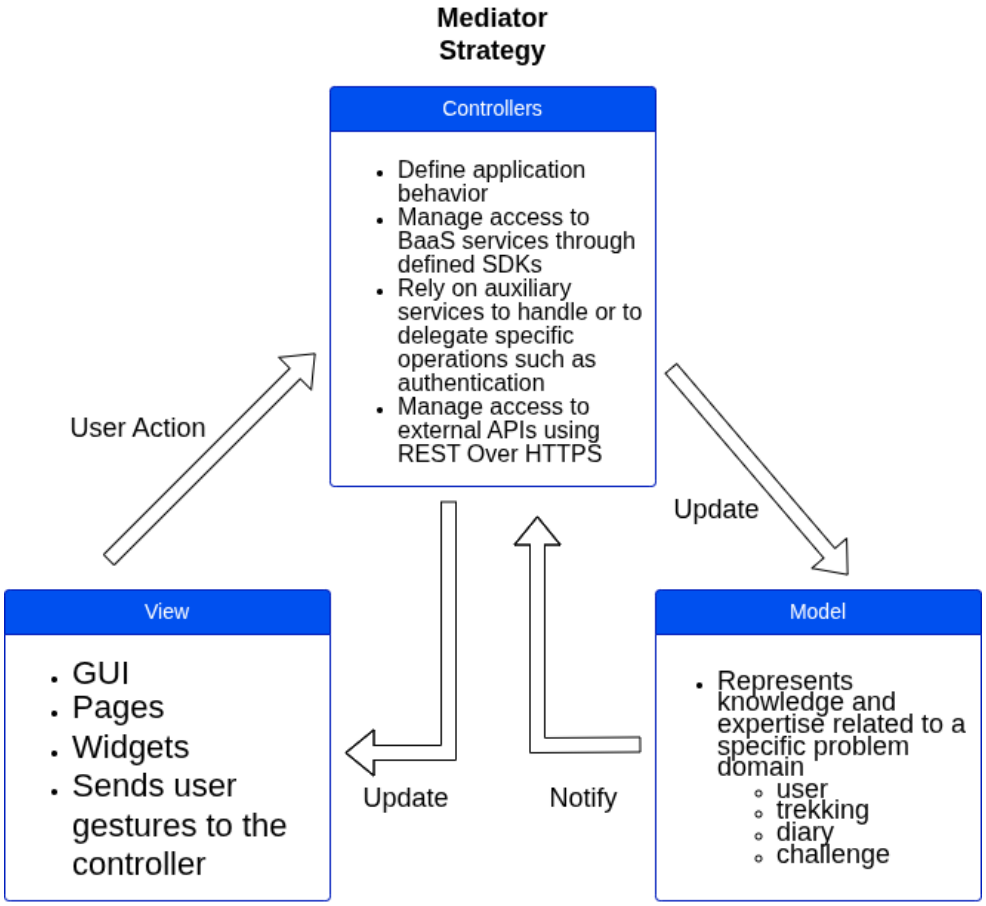


Figure 2.7: Model view controller pattern, adapted from Apple and Oracle documentation

The system adopts the Model–View–Controller (MVC) architectural pattern to separate presentation, application logic, and data management. Within this structure, each role is assigned a specific responsibility in order to improve modularity, maintainability, and clarity of interactions.

The **Model** represents the knowledge and expertise related to the application domain. It includes the core data structures of the system, such as `Diary`, `Trekking`, and `User`. The Model is independent from the user interface and does not handle user interactions directly.

The **View** is responsible for rendering the user interface and collecting user input. Its role is limited to presentation and interaction handling, without directly applying business logic or modifying domain data.

The **Controller** manages the application logic and coordinates the interaction between View and Model. In the adopted variant of MVC, consistent with the interpretation described in both Apple and Oracle documentation, the Controller also plays the role of a *Mediator*. More precisely, the View does not directly manipulate the Model; instead, user actions are forwarded to the Controller, which interprets them, executes the required logic, and updates the corresponding Model data. In this way, the Controller acts as the central coordination point of the interaction, reducing direct dependencies between presentation and domain data.

In addition, the architecture relies on the **Observer pattern** to keep the user interface consistent with state changes. When the state managed by a Controller is updated, the interested Views are notified through the listener mechanism provided by the Flutter framework, based on ChangeNotifier, integrated via Provider, so that they can refresh their representation accordingly. This notification-based interaction avoids tight coupling and supports consistency across multiple dependent Views.

Therefore, the adopted solution combines the structural organization of MVC with the use of the Mediator pattern in the Controller and the Observer pattern. This combination is particularly suitable for the Flutter-based implementation of the system, where the presentation layer, the application logic, and the domain data remain clearly separated while still cooperating through a well-defined interaction model.

2.6.2 ServiceController as a Facade

The ServiceController is designed following the Facade pattern.

It provides a unified interface that aggregates multiple underlying services, including external APIs and OS-level services. Instead of interacting directly with heterogeneous components, UI components and controllers rely on a single access point.

This design reduces coupling and simplifies the interaction model of the system. It also improves readability and consistency, as all non-domain-specific functionalities are accessed through a centralized component.

Furthermore, the facade allows internal changes to the underlying services without impacting the rest of the application, thus increasing flexibility and maintainability.

2.7 Other Design Decisions

2.7.1 Flutter as Application Framework

Flutter was selected for the implementation of the client applications because it provides an effective balance between cross-platform development, interface consistency, and maintainability. A primary requirement of the project was the possibility of deploying the smartphone application on both Android and iOS while preserving the same visual structure and interaction style across platforms. In this respect, Flutter offers a significant advantage, since it enables the development of a single shared codebase while also providing a uniform UI across platforms.

Compared with native alternatives such as Kotlin and Swift, Flutter avoids the need to maintain two separate implementations for Android and iOS, thus reducing duplication of development effort and simplifying maintenance. Compared with other cross-platform solutions, the choice of Flutter was particularly suitable for this project because it supports the construction of a coherent and controlled UI layer, which was important for preserving the same application identity and user experience on both mobile platforms.

2.7.2 Firebase as Backend-as-a-Service

The adoption of a Backend-as-a-Service solution represents a strategic architectural decision aimed at improving maintainability, scalability, and integration.
Firebase was adopted as the backend solution to support a scalable and robust system architecture while maintaining a clear separation between client-side logic and backend responsibilities. Rather than introducing a custom server-side layer, the system relies on a Backend-as-a-Service approach, where core functionalities are delegated to managed cloud services.

This choice allows the architecture to focus on the coordination of application logic on the client side, while relying on well-established and production-ready services for critical backend responsibilities such as authentication, data persistence, and media storage, integrating with the client applications through platform-specific SDKs.

From an architectural perspective, this approach improves modularity and reduces system complexity by eliminating the need to design and maintain a dedicated backend infrastructure. This approach presents notable advantages, including scalability, reliability, and security, through managed services that are designed to handle distributed workloads and concurrent access patterns.

Furthermore, the use of Firebase aligns well with the multi-platform nature of the system, as both smartphone and smartwatch applications can interact with the same backend services in a consistent and unified manner. This contributes to maintaining a coherent data model across different devices.

2.7.3 Phone Smartwatch Pairing

While the smartphone application directly relies on Firebase Authentication, the smartwatch application does not perform authentication independently. Instead, it adopts a pairing mechanism based on QR code scanning. Through this process, the smartphone securely associates an authenticated user with the smartwatch by storing the corresponding identifier in Firestore.
This design choice provides several advantages:

- avoids handling user credentials on the smartwatch or to enforce specific authentication methods, such as Google Sign-In, reducing the exposure of sensitive data on a constrained and potentially less secure device
- simplifies the user interaction model, eliminating the need for credential input on devices where text entry is inherently limited and less user-friendly

For these reasons, the interaction is intentionally defined as a pairing process rather than a traditional login, reflecting both its security properties and its role within the overall system architecture.

2.7.4 OSService Package Design

As shown in the smartphone component diagram (see Figure 2.2.2), the OSService package has been introduced to isolate all interactions with operating system functionalities and external libraries.

In particular, this package includes the following services:

- `PermissionService`, responsible for handling runtime permissions;
- `GeoService`, responsible for accessing device location;
- `MemoryService`, responsible for local storage and caching;
- `NotificationService`, responsible for managing local notifications;

Each service encapsulates the interaction with specific Flutter packages and platform APIs, such as geolocation, local storage, or notification systems. These dependencies correspond to external libraries (e.g., packages available on *pub.dev*) that internally communicate with the underlying operating system.

This design ensures that the rest of the application does not directly depend on platform-specific APIs or external libraries. Instead, all such dependencies are confined within the OSService package.

The main motivation behind this decision is maintainability. If a library or underlying implementation needs to be replaced, only the corresponding service must be modified, without requiring changes in the rest of the system. This significantly reduces the impact of technological changes.

2.7.5 Use of External Service Providers

The application relies on a set of external service providers to implement functionalities related to weather information and map visualization, which are required by several functional requirements of the system. In particular, these providers support the fulfillment of requirements related to the visualization of weather conditions associated with trekking routes, the detection of weather alerts, and the display of geographical information on maps.

From an architectural perspective, the `Service_i` interface acts as the main entry point for all operations involving third-party providers. It exposes methods such as `weather()`, `forecast()`, `weatherbitAlerts()`, and map-related operations like `openTopoMapTile()`.

This design follows a facade-oriented approach: a single interface lets other components access the required functionalities. As a result, other components interact only with abstract operations, without depending on the specific provider for the data. This approach improves modularity and ensures that changes in external services can be handled locally within the service layer, without affecting the rest of the application. Regarding weather data, two different providers have been adopted based on their respective strengths. Standard weather information and forecasts are obtained through OpenWeather, accessed via the `weather()` and `forecast()` methods. These operations are used to

satisfy the functional requirements related to the visualization of current weather conditions and short-term forecasts for trekking locations.

Conversely, weather alerts are handled through Weatherbit, accessed via the `weatherbitAlerts()` operation. This choice is motivated by the availability and reliability of alert-specific data provided by Weatherbit, which are essential to fulfill the functional requirements related to user notification of critical weather conditions.

Map visualization is supported through external tile providers, such as OpenTopoMap, accessed via dedicated operations in `Service_i`. These functionalities are required to satisfy the functional requirements related to the visualization of trekking routes and geographical context. Delegating this responsibility to external providers avoids the need to manage map data internally and significantly reduces system complexity.

Alternative solutions were considered during the design phase. A first option consisted in adopting a single weather provider for both standard weather data and alerts. However, this approach was discarded due to limitations in the quality, availability, or completeness of alert data offered by individual providers. In particular, while OpenWeather provides reliable general weather information, its alert coverage is not always sufficient for the requirements of the application.

The chosen solution, based on multiple providers integrated through a unified interface, such as `Service_i`, represents a trade-off between flexibility and architectural simplicity. It allows the system to leverage the strengths of different providers while keeping responsibilities well separated and interactions with higher level components stable.

Overall, the use of external service providers, combined with the abstraction introduced by the `Service_i` interface, enables the system to satisfy the relevant functional requirements while preserving modularity, maintainability, and extensibility of the architecture.

2.7.6 Network Connectivity Requirements and smartwatch Platform

The smartwatch application requires reliable access to internet connectivity in order to provide a complete and consistent user experience. In particular, network access is necessary for several core functionalities of the system.

The application relies on external services to retrieve real-time data, such as maps, which are essential for trekking activities.

Internet access enables communication with backend services and APIs, allowing the application to fetch dynamic content and maintain updated information.

Given these requirements, the choice of Wear OS as the target platform is appropriate and justified. Wear OS supports multiple network communication modes, including:

- **Bluetooth connectivity**: allows the smartwatch to access the internet indirectly through a paired smartphone.
- **Wi-Fi connectivity**: enables the device to connect directly to wireless networks, allowing standalone operation in specific scenarios.
- **Cellular connectivity (LTE with eSIM)**: available on selected devices, providing full independence from the smartphone and ensuring continuous connectivity during outdoor activities.

This flexibility ensures that the application can operate under different conditions, ranging from smartphone-assisted usage to fully autonomous scenarios. In particular, the availability of LTE-enabled devices makes Wear OS especially suitable for outdoor and trekking contexts, where the user may not always have access to their smartphone.

3. Functional Requirements

3.1 Smartphone Application Functional Requirements

3.1.1 Account and Profile Management

FR1 The system shall allow users to register by providing email and password

FR2 The system shall allow users to register using an external authentication provider

FR3 The system shall allow users to log in using email and password

FR4 The system shall allow users to log in using an external authentication provider

FR5 The system shall allow users to reset their password

FR6 When registration is successful, the system shall create a user profile

FR7 When registration is successful, the system shall assign a default username based on the user's email

FR8 The system shall allow users to update their username

FR9 The system shall allow users to update their profile image

FR10 The system shall allow users to update their name

FR11 The system shall allow users to update their surname

FR12 The system shall allow users to update their birth date

FR13 When users are registered with email and password, the system shall allow them to update their email address

FR14 When users are authenticated through an external provider, the system shall allow them to restore their profile image from the provider

FR15 The system shall allow users to change the application language

FR16 The system shall allow users to pair a smartwatch with their account

FR17 The system shall allow users to enable weather notifications for selected routes

3.1.2 Home, Routes and Route Details

FR18 The system shall display routes on a map

FR19 The system shall classify routes according to their difficulty level

FR20 The system shall display routes using different colors based on their difficulty level

FR21 When a user selects a route, the system shall display detailed information about the selected route

FR22 The system shall display route information including starting point, ending point, difficulty level, length, estimated time, and elevation gain

FR23 The system shall indicate the suitability of a route for families

FR24 The system shall display a textual description of the route

FR25 The system shall display refreshment points associated with the route

FR26 The system shall display challenges associated with the route

FR27 The system shall display summary weather information for the selected route

FR28 The system shall allow users to save a route to their profile

FR29 The system shall allow users to remove a saved route from their profile

FR30 The system shall allow users to start a navigation session for a selected route

FR31 When a navigation session ends, the system shall suggest creating a diary entry for the route

3.1.3 Diary Management

FR32 The system shall allow users to create a diary entry associated with a selected route

FR33 The system shall allow users to specify the date of the activity in a diary entry

FR34 The system shall allow users to specify the duration of the activity in a diary entry

FR35 The system shall allow users to select followed users who participated in the activity

FR36 The system shall allow users to add textual notes to a diary entry

FR37 The system shall allow users to indicate the presence of refreshment stops during the activity

FR38 The system shall allow users to select completed challenges in a diary entry

FR39 The system shall allow users to select a mood associated with the activity

FR40 The system shall allow users to upload images to a diary entry

FR41 The system shall allow users to mark a diary entry as public or private

FR42 The system shall allow users to save a diary entry

FR43 The system shall allow users to discard a diary entry before saving it

3.1.4 Search and Challenges

FR44 The system shall allow users to search for other users

FR45 The system shall allow users to search for routes

FR46 The system shall display search results according to the selected search type

FR47 The system shall display public profile information of searched users

FR48 The system shall display route information for searched routes

FR49 The system shall display diary entries from followed users

FR50 The system shall display the available challenges

FR51 The system shall display the requirements for completing each challenge

3.1.5 Profile and Friends Features

FR52 The system shall display the user's profile picture, username, and level

FR53 The system shall display the number of diary entries created by the user

FR54 The system shall display the number of followers of the user

FR55 The system shall display the number of followed users

FR56 The system shall allow users to access the public profile of a follower

FR57 The system shall allow users to access the public profile of a followed user

FR58 The system shall allow users to share a public link to their profile

3.1.6 Navigation and Sensor Information

FR59 The system shall display compass information

FR60 The system shall display the current heading in degrees

FR61 The system shall display the user's current altitude

FR62 The system shall display the user's current geographic coordinates

3.1.7 Weather Information

FR63 The system shall allow users to access detailed weather information for a selected route

FR64 The system shall display a five-day weather forecast related to the selected route

FR65 The system shall allow users to request weather information using their current GPS position

FR66 When the user selects route-based weather information, the system shall display weather data for the route area

FR67 When the user selects GPS-based weather information, the system shall display weather data for the user's current position

FR68 The system shall display a map preview associated with the currently selected weather location

FR69 The system shall display weather alerts related to the currently selected weather location

FR70 The system shall allow users to enable weather alert notifications for a selected route

FR71 The system shall allow users to access a satellite map view displaying both the selected route position and the user's current position, using distinct visual indicators

FR72 The system shall allow users to display weather layers on the map

FR73 The system shall allow users to display precipitation information on the map

FR74 The system shall allow users to display snow information on the map

FR75 The system shall allow users to display wind information on the map

FR76 The system shall allow users to display cloud information on the map

FR77 The system shall allow users to display temperature information on the map

FR78 The system shall allow users to display pressure information on the map

FR79 The system shall display a legend associated with the selected weather layer

3.1.8 Offline mode

FR80 The system shall detect when the internet connection is not available

FR81 When the internet connection is not available, the system shall display a dedicated offline page

FR82 In offline mode, the system shall allow users to access compass functionalities

FR83 In offline mode, the system shall allow users to access their current GPS position

FR84 In offline mode, the system shall allow users to access altitude information

FR85 When the internet connection is restored, the system shall allow users to resume online functionalities

3.2 Smartwatch Application Functional Requirements

3.2.1 Access & Synchronization

FR1 The system shall allow users to access smartwatch functionalities after pairing the device with their account.

FR2 The system shall synchronize route and user data with the associated smartphone application.

FR3 When synchronization fails, the system shall notify the user.

FR4 The system shall allow users to enable weather notifications for selected routes.

3.2.2 Routes & Map Visualization

FR5 The system shall display routes on a smartwatch map.

FR6 The system shall display refuge points as spots on the route.

FR7 The system shall allow users to select a refuge point on the map.

FR8 When a refuge point is selected, the system shall display associated information.

FR9 The system shall allow users to scroll through route-related information.

FR10 The system shall classify routes according to their difficulty level.

FR11 The system shall display routes using different colors based on their difficulty level.

3.2.3 Route Details

FR12 The system shall display key information about a selected route.

FR13 When a user selects a route, the system shall display detailed information about the selected route.

FR14 The system shall display route information including difficulty level, distance, estimated time, and elevation gain.

FR15 The system shall indicate the suitability of a route for families.

FR16 The system shall display a concise textual description of the route.

FR17 The system shall display refreshment points associated with the route.

FR18 The system shall display challenges associated with the route.

FR19 The system shall display summary weather information for the selected route.

FR20 The system shall allow users to start a navigation session for a selected route.

3.2.4 Weather Information

FR21 The system shall display weather information for a selected route.

FR22 The system shall display current weather conditions.

FR23 The system shall display a short-term forecast.

FR24 When weather information is unavailable, the system shall notify the user.

FR25 The system shall allow users to receive weather alert notifications for selected routes.

FR26 The system shall display weather alerts related to the currently selected weather location.

FR27 The system shall allow users to enable weather alert notifications for a selected route.

3.2.5 Friends' Routes

FR28 The system shall display routes published by followed users.

FR29 The system shall allow users to browse routes shared by friends.

FR30 The system shall allow users to select a route shared by a followed user.

FR31 The system shall display basic information about routes shared by friends.

3.2.6 Interaction & Smartwatch Constraints

FR32 The system shall present information in a format optimized for a small screen.

FR33 The system shall allow users to navigate content through touch-based scrolling.

FR34 The system shall minimize user interaction steps to access route information.

FR35 The system shall maintain the selected route context during navigation between views.